

Proyecto piedra azul

Descripción de la Arquitectura de Software

Autores

*Juan Camilo Meneses Daza
Juan Felipe Ramírez Salazar
José Nicolas Bambague Sánchez
Juan Alejandro Cardenas*

Portada

Historial de Revisión

Versión	Autores	Fecha
0.1		Mayo 2023
0.31	<i>Francisco Obando</i>	Mayo 2023

Tabla de Contenido

- **Historial de Revisión**
- **Portada**
- **Página**
- 1. **Introducción (4)**
- 2. **Descripción del problema (4)**
- 3. **Metas y Restricciones de la Arquitectura (5)**
 - **3.1 Metas arquitectónicas (5)**
 - **3.2 Restricciones arquitectónicas (6)**
 - **3.3 Resumen de metas y restricciones (7)**
- 4. **Historias de usuario (7)**
- 5. **Prototipos de la interfaz de usuario y un test de evaluación de usabilidad (8)**
 - **HU-1 – Listado de citas por médico y fecha (9)**
 - **HU-2 – Crear nueva cita (agendador) (9)**
 - **HU-3 – Reserva de cita por el paciente (portal público) (10)**
 - **HU-4 – Configuración de disponibilidad por profesional (administrador) (11)**
 - **HU-5 – Exportación de citas en formato CSV/Excel (12)**
 - **Test de Evaluación de Usabilidad (13)**
 - **Resumen General de Usabilidad (14)**
- 6. **Pantallazo de planificación de tareas del Sprint 2: Jira, Trello. (15)**
- 7. **Escenarios de calidad del atributo de seguridad (16)**
 - **QS1 Acceso no autorizado al portal interno (17)**
 - **QS2 Protección contra ataques de fuerza bruta (18)**
 - **Resumen de Escenarios de Seguridad (19)**
- 8. **Arquitectura y diseño de software usando el modelo C4 y 4+1 vistas. (20)**
 - **8.1 Modelo C4 (20)**
 - **8.1.1 Diagrama de Sistema (System Landscape) (21)**
 - **8.1.2 Diagrama de Contexto (System Context – Nivel 1) (21)**
 - **8.1.3 Diagrama de Contenedores (Nivel 2) (22)**
 - **8.1.4 Diagrama de Componentes (Nivel 3) API REST (23)**
 - **8.1.5 Diagrama de Despliegue (Deployment) (24)**
 - **8.1.6 Diagrama Dinámico 1 Paciente reserva una cita médica (RF3) (25)**
 - **8.1.7 Diagrama Dinámico 2 – Reagendamiento con historial (RF6) (26)**
 - **8.2 Vistas 4+1 (27)**
 - **8.2.1 Vista +1: Escenarios Diagrama de Casos de Uso (28)**
 - **8.2.2 Vista Lógica Diagrama de Clases: Gestión de Citas (29)**
 - **8.2.3 Vista Lógica Diagrama de Clases: Gestión de Pacientes (30)**
 - **8.2.4 Vista de Procesos Diagrama de Secuencia: Reservar Cita (31)**
 - **8.2.5 Vista de Procesos – Diagrama de Secuencia: Reagendar Cita (32)**
- 9. **Listado de patrones de diseño GoF (33)**
 - **Patrones Creacionales (33)**
 - **Patrones Estructurales (33)**
 - **Patrones de Comportamiento (34)**
 - **En el Frontend (React) (35)**
- 10. **Rationale (36)**

Proyecto Piedra Azul

Architecture Document – SAD

1 Introducción

El presente documento describe la arquitectura y el diseño de software del sistema **Piedrazul**, una aplicación web de reserva de citas médicas desarrollada para el Centro Médico de Medicina Alternativa Piedrazul. Este documento corresponde a la **segunda iteración** del proyecto de curso de la asignatura Ingeniería de Software III de la Universidad del Cauca, periodo 2026-1.

El sistema Piedrazul es una Single Page Application (SPA) construida con **React + Vite** en el frontend y una **API REST con ASP.NET Core** en el backend, conectada a una base de datos **PostgreSQL**. La autenticación está integrada con **Keycloak** como proveedor de identidad basado en los estándares OAuth 2.0 y OpenID Connect.

El propósito principal del sistema es digitalizar el proceso de agendamiento de citas del centro médico, que anteriormente se gestionaba de forma manual a través de WhatsApp y registros físicos. La solución permite que los pacientes reserven citas de forma autónoma desde la web, que los agendadores gestionen citas de pacientes que contactan por teléfono o WhatsApp, y que el administrador configure la disponibilidad de cada profesional.

El documento está organizado de la siguiente manera: se presentan las historias de usuario implementadas en esta iteración con sus criterios de aceptación, los prototipos de interfaz con su evaluación de usabilidad, la planificación del sprint, los escenarios de calidad del atributo de seguridad, la arquitectura del sistema usando el modelo C4 y las vistas 4+1, los patrones de diseño GoF aplicados, y finalmente los enlaces al repositorio y al video de sustentación.

2 Descripción del problema

La Red de Servicios Médicos de Piedrazul, ubicada en Popayán, presta servicios de salud enfocados en medicina alternativa, entre ellos terapia neural, quiropráctica y fisioterapia. Debido al crecimiento en la demanda de atención médica, el proceso de asignación de citas se ha convertido en una actividad crítica para la operación diaria del centro médico.

Inicialmente, Piedrazul no contaba con un sistema formal de asignación de citas. Los pacientes asistían directamente a las instalaciones y esperaban su turno de atención. Este modelo funcionaba mientras el volumen de usuarios era bajo; sin embargo, con el aumento progresivo de pacientes, comenzaron a presentarse largas esperas, desorganización en la jornada de atención y dificultades para planificar adecuadamente el trabajo de médicos y terapeutas.

Posteriormente, el centro adoptó mecanismos de gestión más estructurados, como el uso de hojas de cálculo y una aplicación de escritorio para registrar pacientes, asignar citas, consultar agendas y generar listados diarios. Aunque esta solución permitió mejorar el control operativo frente al proceso manual inicial, también introdujo nuevas limitaciones. Entre ellas se encuentran la dependencia del personal médico o administrativo para registrar cada cita, la posibilidad de errores humanos, la duplicidad de información, la ausencia de un canal de autogestión para pacientes y la dificultad para escalar el sistema ante un mayor número de usuarios.

El proceso actual de agendamiento depende en gran medida de llamadas telefónicas o mensajes por WhatsApp. Los médicos o agendadores deben recibir la solicitud, recopilar los datos básicos del paciente, revisar la disponibilidad y registrar manualmente la cita. Esta dinámica consume tiempo operativo, especialmente en horarios de alta demanda, y limita la capacidad del centro para atender de

forma eficiente nuevas solicitudes.

Adicionalmente, la aplicación de escritorio existente presenta una arquitectura monolítica, lo cual dificulta la evolución del sistema hacia nuevas funcionalidades como el agendamiento autónomo por parte del paciente, la autenticación segura por roles, la trazabilidad de acciones, la auditoría, la exportación flexible de citas y la integración futura con otros servicios.

Por lo anterior, se requiere una aplicación web que permita modernizar el proceso de gestión de citas médicas, habilitando el agendamiento autónomo y manual, la configuración de disponibilidad por profesional, la consulta y exportación de agendas, y el control seguro de acceso según roles. La solución debe estar soportada por una arquitectura mantenible, segura y evolutiva, capaz de responder a los requisitos funcionales y no funcionales definidos para el sistema Piedrazul.

3 Metas y Restricciones de la Arquitectura

La arquitectura del sistema Piedrazul se define a partir de las necesidades funcionales del centro médico, las restricciones académicas del proyecto y los atributos de calidad requeridos para soportar una solución web segura, mantenible y evolutiva. Esta sección establece las metas arquitectónicas y las restricciones que guían las decisiones de diseño del sistema.

3.1 Metas arquitectónicas

Las metas arquitectónicas representan los objetivos principales que la arquitectura debe satisfacer. Para Piedrazul, estas metas se derivan del contexto del problema, de los requisitos funcionales y de los requisitos no funcionales definidos para el proyecto.

Seguridad y control de acceso.

El sistema debe proteger la información de pacientes, citas e historias clínicas mediante autenticación segura y autorización basada en roles. La arquitectura debe impedir que usuarios no autenticados o con permisos insuficientes accedan a funcionalidades internas como gestión de citas, configuración de disponibilidad o exportación de información. Esta meta se relaciona directamente con la necesidad de restringir el acceso a funcionalidades según el rol asignado a cada usuario.

Modificabilidad y mantenibilidad.

El sistema debe facilitar la incorporación de nuevas funcionalidades sin afectar de forma significativa los módulos existentes. Esto es especialmente importante porque Piedrazul requiere evolucionar desde una aplicación de escritorio monolítica hacia una solución web moderna. Por esta razón, la arquitectura debe separar adecuadamente la lógica de negocio, la infraestructura, la presentación y los mecanismos de persistencia.

Usabilidad.

La solución debe ofrecer una interfaz clara y sencilla para usuarios con distintos niveles de conocimiento técnico, incluyendo pacientes, médicos, terapeutas, agendadores y administradores. La arquitectura debe soportar flujos simples para agendar citas, consultar disponibilidad y registrar información, reduciendo la carga operativa del personal de Piedrazul.

Consistencia de datos y control de concurrencia.

El sistema debe garantizar que dos usuarios no puedan reservar la misma franja horaria para el mismo profesional. La arquitectura debe contemplar validaciones de negocio, restricciones de persistencia y manejo transaccional para evitar duplicidad de citas e inconsistencias en la agenda médica.

Trazabilidad y auditoría.

Las acciones relevantes del sistema deben quedar registradas, incluyendo agendamiento, re-agendamiento, modificación de usuarios, cambios de estado y acceso a información sensible. Esta meta permite realizar seguimiento operativo y verificar quién realizó cada acción dentro del sistema.

Escalabilidad y evolución futura.

La Arquitectura debe permitir el crecimiento del sistema sin rediseñar su estructura base. Aunque el alcance actual corresponde a un contexto académico y a una organización específica, el diseño debe contemplar la evolución hacia nuevas funcionalidades, mayor número de usuarios y una posible adaptación futura a múltiples organizaciones de salud.

Portabilidad del despliegue.

El sistema debe poder desplegarse de forma reproducible en distintos entornos de desarrollo o ejecución. Para ello, la arquitectura debe favorecer el uso de contenedores, configuración externa y tecnologías compatibles con sistemas operativos modernos.

3.2 Restricciones arquitectónicas

Las restricciones arquitectónicas delimitan las decisiones técnicas, funcionales y de alcance que deben respetarse durante el diseño e implementación del sistema.

Tipo de aplicación.

La solución debe implementarse como una aplicación web. El proyecto permite enfoques MPA, SPA o PWA, pero la elección debe justificarse según los requisitos funcionales y no funcionales del sistema. Para Piedrazul se adopta una **Single Page Application (SPA)**, dado que permite una experiencia fluida para el usuario, navegación rápida entre módulos y actualización dinámica de información como disponibilidad de citas, formularios y listados.

Tecnologías de frontend.

El frontend se desarrolla con **React, TypeScript y Vite**, tecnologías adecuadas para construir interfaces web modernas, modulares y reutilizables. Esta elección facilita la separación de componentes, la validación de formularios, la creación de flujos guiados de agendamiento y la evolución progresiva del portal público e interno.

Tecnologías de backend.

El backend se implementa como una **API REST con ASP.NET Core**, lo cual permite exponer servicios HTTP para los diferentes portales del sistema: portal público, portal interno, administración y funcionalidades de paciente. Esta decisión favorece una arquitectura desacoplada entre cliente y servidor, facilitando pruebas, mantenimiento y evolución independiente de cada capa.

Base de datos.

El sistema utiliza **PostgreSQL** como motor de base de datos relacional. Esta elección responde a la necesidad de garantizar integridad, consistencia y persistencia de datos críticos como pacientes, citas, disponibilidad de profesionales, usuarios, auditoría e información clínica. El documento del proyecto exige que la base de datos seleccionada soporte persistencia de datos críticos, concurrencia y crecimiento futuro.

Autenticación y autorización.

La autenticación y autorización se integran con **Keycloak**, utilizando tokens JWT. Esta restricción responde al requisito no funcional del segundo corte, que solicita implementar autenticación y autorización basada en JWT mediante un sistema externo como Keycloak. El uso de Keycloak permite centralizar usuarios, roles y emisión de tokens, reduciendo el acoplamiento entre la lógica de negocio y la gestión de identidad.

Arquitectura interna del backend.

El backend sigue el enfoque de **Clean Architecture**, separando el sistema en capas de dominio, aplicación, infraestructura y presentación. Esta decisión permite que la lógica de negocio permanezca independiente de frameworks, base de datos, autenticación o mecanismos externos, favoreciendo pruebas unitarias y modificabilidad.

Alcance académico.

La solución se desarrolla dentro del contexto académico del curso. Por tanto, no es obligatorio implementar integraciones externas reales como WhatsApp, SMS o pasarelas de pago. Estas funcionalidades pueden simularse o dejarse preparadas arquitectónicamente para futuras iteraciones, siempre que se justifique su diseño.

Disponibilidad de tiempo y equipo.

El desarrollo está limitado por la duración del segundo corte y por la capacidad del equipo de trabajo. Por esta razón, la arquitectura debe balancear calidad técnica con viabilidad de implementación, priorizando los requisitos funcionales de mayor valor para el cliente y los requisitos no funcionales evaluados en la iteración.

Privacidad de la información clínica.

El sistema debe proteger los datos personales y clínicos de los pacientes. El acceso a esta información debe restringirse por rol, y toda consulta o modificación debe quedar registrada mediante mecanismos de auditoría. Esto condiciona el diseño de los módulos de autenticación, autorización, auditoría y persistencia.

3.3 Resumen de metas y restricciones

Elemento	Decisión arquitectónica asociada
Seguridad	Keycloak, JWT, autorización por roles
Modificabilidad	Clean Architecture, separación por capas
Usabilidad	SPA con React, flujos guiados
Consistencia	PostgreSQL, validaciones de negocio y transacciones
Trazabilidad	Auditoría con registro de acciones relevantes
Escalabilidad	API REST desacoplada, Redis y diseño evolutivo
Portabilidad	Docker y configuración por entorno
Alcance académico	Simulación de integraciones externas no obligatorias

4. Vista de Requisitos Funcionales: Historias de Usuario

Esta sección presenta las historias de usuario trabajadas para la segunda iteración del sistema Piedrazul. Las historias fueron documentadas en la plantilla de historias de usuario del equipo, estructuradas mediante épica, identificador de historia, rol, funcionalidad esperada, razón de negocio y criterios de aceptación expresados en formato de escenarios.

Las historias documentadas corresponden a los requisitos funcionales priorizados para el segundo corte: listado de citas por médico y fecha, creación manual de citas por parte del agendador, agendamiento autónomo desde la web, configuración de parámetros del sistema de agendamiento y exportación de citas en formato CSV.

4.1 Resumen de historias de usuario

ID épica	ID historia	Rol	Funcionalidad	Resultado esperado
HE-01	HU-1	Agendador de citas	Listar las citas médicas de un médico en una fecha específica	Visualizar el listado de citas programadas y conocer cuántas citas tiene el médico ese día.
HE-01	HU-2	Agendador de citas	Crear una nueva cita para un paciente que contactó por WhatsApp	Registrar la cita en el sistema y asegurar que el paciente tenga su turno asignado correctamente.
HE-01	HU-3	Paciente	Agendar una cita desde la web	Obtener una cita de forma rápida y sin necesidad de contactar al agendador.
HE-01	HU-4	Administrador	Configurar los parámetros del sistema de agendamiento	Controlar la disponibilidad de los médicos y garantizar que las citas se generen correctamente según los horarios definidos.
HE-01	HU-5	Médico / terapeuta / agendador	Exportar las citas de un médico en una fecha específica en formato CSV	Obtener un listado imprimible u organizable en hoja de cálculo

4.2 HU-1 Listado de citas por médico/terapeuta y fecha

Épica: HE-01

Historia: HU-1

Rol: Yo como agendador de citas

Quiero: listar las citas médicas de un médico en una fecha específica.

Para: visualizar el listado de citas programadas y conocer cuántas citas tiene el médico ese día.

Criterios de aceptación:

Escenario	Criterio	Contexto	Evento	Resultado esperado
1	Búsqueda por médico y fecha	Dado que el agendador está en el módulo de consulta de citas	Cuando selecciona un médico y una fecha	Entonces el sistema debe mostrar todas las citas agendadas para ese médico en esa fecha.
2	Visualización en tabla	Dado que existen citas registradas para el médico en la fecha seleccionada	Cuando se realiza la búsqueda	Entonces el sistema debe mostrar los resultados en una tabla con: hora de la cita, nombre del paciente, número de documento y estado de la cita.
3	Conteo de citas	Dado que se muestran las citas en la tabla	—	Entonces el sistema debe mostrar el total de citas encontradas.
4	Sin resultados	Dado que no existen citas para el médico en esa fecha	Cuando se realiza la búsqueda	Entonces el sistema debe mostrar un mensaje indicando que no hay citas registradas.

Relación arquitectónica:

Esta historia requiere una consulta filtrada por médico y fecha desde la API REST. La responsabilidad de búsqueda se separa entre el frontend, que captura los filtros, la capa de aplicación, que coordina la consulta, y la capa de infraestructura, que obtiene la información desde la base de datos.

4.3 HU-2 Crear nueva cita desde el portal interno

Épica: HE-01

Historia: HU-2

Rol: Yo como agendador de citas

Quiero: crear una nueva cita para un paciente que contactó por WhatsApp.

Para: registrar la cita en el sistema y asegurar que el paciente tenga su turno asignado correctamente.

Criterios de aceptación:

Escenario	Criterio	Contexto	Evento	Resultado esperado
1	Registro de datos del paciente	Dado que el agendador desea crear una cita	Cuando ingresa los datos del paciente	Entonces el sistema debe permitir registrar: número de documento, nombres, apellidos, celular, género, fecha de nacimiento opcional y correo electrónico opcional.
2	Selección de médico y hora	Dado que el paciente fue registrado	Cuando el agendador selecciona un médico y una hora disponible	Entonces el sistema debe permitir agendar la cita.
3	Validación de disponibilidad	Dado que el agendador intenta registrar una cita	Cuando la hora ya está ocupada	Entonces el sistema debe mostrar un mensaje indicando que la hora no está disponible.
4	Confirmación de cita	Dado que todos los datos son válidos	Cuando el agendador confirma la cita	Entonces el sistema debe guardar la cita y mostrar un mensaje de confirmación.

Relación arquitectónica:

Esta historia exige validaciones de disponibilidad antes de guardar la cita. La lógica de negocio no debe depender del formulario del frontend; debe estar ubicada en servicios de aplicación que verifiquen disponibilidad, datos del paciente y consistencia de la agenda.

4.4 HU-3 Agendar cita desde el portal web

Épica: HE-01

Historia: HU-3

Rol: Yo como paciente

Quiero: agendar una cita desde la web.

Para: obtener una cita de forma rápida y sin necesidad de contactar al agendador.

Criterios de aceptación:

Escenario	Criterio	Contexto	Evento	Resultado esperado
1	Registro de usuario	Dado que el paciente no tiene cuenta	Cuando intenta agendar una cita	Entonces el sistema debe solicitar el registro de usuario.
2	Visualización de médicos	Dado que el paciente está autenticado	Cuando accede al módulo de agendamiento	Entonces el sistema debe mostrar la lista de médicos disponibles.
3	Visualización de horarios disponibles	Dado que el paciente selecciona un médico	—	Entonces el sistema debe mostrar las franjas horarias disponibles.
4	Confirmación de cita	Dado que el paciente selecciona una franja disponible	Cuando confirma la cita	Entonces el sistema debe registrar la cita y mostrar un mensaje de confirmación.

Relación arquitectónica:

Esta historia justifica el uso de una aplicación web tipo SPA, porque el paciente necesita un flujo simple y guiado. La disponibilidad de horarios debe calcularse desde el backend para evitar inconsistencias o manipulación desde el cliente.

4.5 HU-4 Configuración de disponibilidad de profesionales

Épica: HE-01

Historia: HU-4

Rol: Yo como administrador

Quiero: configurar los parámetros del sistema de agendamiento.

Para: controlar la disponibilidad de los médicos y garantizar que las citas se generen correctamente según los horarios definidos.

Criterios de aceptación:

Escenario	Criterio	Contexto	Evento	Resultado esperado
1	Configurar ventana de agendamiento	Dado que el administrador accede al módulo de configuración	Cuando define el número de semanas disponibles para citas	Entonces el sistema debe guardar la configuración.
2	Configurar días de atención	Dado que el administrador selecciona un médico	Cuando define los días de atención	Entonces el sistema debe registrar esa disponibilidad.
3	Configurar franja horaria	Dado que el administrador configura un médico	Cuando define la hora de inicio y fin de atención	Entonces el sistema debe guardar la franja horaria.
4	Configurar intervalo entre citas	Dado que el administrador configura un médico	Cuando define el intervalo en minutos entre citas	Entonces el sistema debe aplicar ese intervalo al generar los horarios.

Relación arquitectónica:

Esta historia impacta directamente el cálculo de disponibilidad. Los parámetros configurados por el administrador deben persistirse y ser consumidos por los servicios encargados de generar las franjas horarias disponibles para pacientes y agendadores.

4.6 HU-5 Exportación de citas por médico/terapeuta y fecha

Épica: HE-01

Historia: HU-5

Rol: Yo como médico, terapeuta o agendador

Quiero: exportar las citas de un médico en una fecha específica en formato CSV.

Para: tener un listado imprimible u organizable en hoja de cálculo durante la jornada de atención.

Criterios de aceptación:

Escenario	Criterio	Contexto	Evento	Resultado esperado
1	Selección de parámetros	Dado que el agendador o médico está en el módulo de citas	Cuando selecciona un médico y una fecha y hace clic en exportar	Entonces el sistema genera un archivo CSV con las citas de ese médico en esa fecha.
2	Contenido del archivo	Dado que el archivo CSV fue generado exitosamente	Cuando el usuario lo abre en una hoja de cálculo	Entonces el archivo debe contener las columnas: hora, nombre del paciente, documento, celular, especialidad, observaciones y estado.
3	Sin citas	Dado que no hay citas para el médico en la fecha seleccionada	Cuando el usuario intenta exportar	Entonces el sistema informa que no hay citas para exportar y no genera el archivo.
4	Control de acceso	Dado que un paciente intenta acceder a la exportación	Cuando realiza la petición	Entonces el sistema deniega el acceso con error 403.

Relación arquitectónica:

Esta historia requiere control de acceso por rol y generación de archivos desde el backend. La exportación no debe estar disponible para pacientes, ya que contiene información operativa y datos de otros usuarios del sistema.

4.7 Observaciones arquitectónicas de las historias

Las historias implementadas en la segunda iteración evidencian tres necesidades arquitectónicas principales:

1. **Separación de responsabilidades:** las reglas de negocio de citas, disponibilidad y exportación no deben depender de la interfaz gráfica ni de detalles de infraestructura.
2. **Seguridad transversal:** las funcionalidades internas deben estar protegidas mediante autenticación y autorización por roles.
3. **Consistencia operacional:** el sistema debe evitar duplicidad de citas, validar disponibilidad y registrar acciones relevantes para auditoría.

Estas necesidades justifican la adopción de una arquitectura basada en API REST, Clean Architecture, autenticación con Keycloak/JWT y persistencia relacional con PostgreSQL.

[Historias de usuario](#)

Este recurso permite revisar cada historia, sus escenarios y criterios de validación de forma estructurada, facilitando la colaboración y toma de decisiones dentro del equipo de trabajo.

5. Prototipos de Interfaz de Usuario y Evaluación de Usabilidad

Esta sección presenta los prototipos funcionales de interfaz desarrollados para las historias de usuario documentadas en la plantilla de trabajo del equipo. Los prototipos corresponden a las historias HU-1, HU-2, HU-3, HU-4 y HU-5, las cuales fueron priorizadas para la segunda iteración del sistema Piedrazul.

Los prototipos fueron contruidos como interfaces web dentro del frontend del sistema, utilizando React, TypeScript y Vite. Su objetivo principal es validar los flujos de interacción asociados al listado de citas, creación manual de citas, agendamiento autónomo desde la web, configuración de parámetros del sistema y exportación de citas en formato CSV.

El diseño de las interfaces se orientó a usuarios con distintos roles dentro del sistema: agendador de citas, paciente, administrador, médico y terapeuta. Por esta razón, se priorizó el uso de lenguaje claro, formularios simples, pasos guiados, retroalimentación visual y controles reconocibles como listas desplegables, calendarios, botones de acción y tablas de resultados.

La evaluación de usabilidad se realizó mediante inspección heurística basada en los diez principios de Jakob Nielsen. Para cada prototipo se revisaron criterios como visibilidad del estado del sistema, correspondencia con el mundo real, control del usuario, consistencia, prevención de errores, reconocimiento antes que recuerdo, flexibilidad, diseño estético, recuperación ante errores y ayuda contextual.

5.1 Resumen de prototipos evaluados

ID	Prototipo	Historia relacionada	Actor principal	Objetivo de usabilidad
P-1	Listado de citas por médico y fecha	HU-1	Agendador de citas	Permitir consultar rápidamente las citas programadas de un médico en una fecha específica y visualizar el total de citas encontradas.
P-2	Creación manual de cita desde el portal interno	HU-2	Agendador de citas	Permitir registrar una cita solicitada por WhatsApp, capturando los datos del paciente y validando la disponibilidad del médico y la hora.
P-3	Agendamiento de cita desde la web	HU-3	Paciente	Permitir que un paciente registrado y autenticado pueda seleccionar un médico, visualizar horarios disponibles y confirmar una cita.
P-4	Configuración de parámetros del sistema de agendamiento	HU-4	Administrador	Permitir configurar la ventana de agendamiento, días de atención, franjas horarias e intervalo entre citas por médico.
P-5	Exportación de citas en formato CSV	HU-5	Médico / Terapeuta / Agendador	Permitir generar un archivo CSV con las citas de un médico en una fecha específica para usarlo en hoja de cálculo o impresión.

5.2 Prototipo H-1 — Listado de citas por médico/terapista y fecha

Listado de citasNueva cita

☐ Seleccionar varias fechas

Profesional

Medicina general - Ana Gómez

Fecha

04/05/2026

Buscar

Descargar PDF

CSV

Excel

Ana Gómez

Medicina general · 4 de mayo de 2026

Total de citas: 1

Hora	Paciente	Documento	Celular	Canal	Estado	Actualizar estado
08:00 - 08:30	Paciente Demo	1000000001	3001234567	Web	Programada	Programada Guardar Antes de la hora de atención solo puedes cambiar la cita a Cancelada.

Crear nueva cita

Descripción del prototipo:

La pantalla principal del portal interno presenta dos pestañas en la parte superior: "Listado de citas" y "Nueva cita". En la sección de listado, el agendador puede seleccionar un profesional desde un dropdown y una fecha desde un selector de calendario. Los botones de acción disponibles son: Buscar, Descargar PDF, CSV y Excel, permitiendo exportar el resultado en múltiples formatos. Una vez realizada la búsqueda, se muestra una tabla con las columnas: Hora, Paciente, Documento, Celular, Canal, Estado y Actualizar estado. Cada fila permite cambiar el estado de la cita directamente desde la tabla con un dropdown y un botón Guardar. En la parte inferior se muestra el total de citas encontradas y un botón para crear una nueva cita.

Elementos destacados:

- Checkbox "Seleccionar varias fechas" para consultas de rango.
- Restricción de cambio de estado: antes de la hora de atención solo se puede cancelar.
- Indicador de total de citas en la esquina superior derecha del resultado.

Relación con la HU-1:

Este prototipo responde a los criterios de búsqueda por médico y fecha, visualización en tabla, conteo de citas y mensaje informativo cuando no existen resultados.

5.3 Prototipo H-2 — Creación de cita desde portal interno



Piedrazul
Centro médico

Portal interno

Agendador Piedrazul
Personal autorizado

Cerrar sesión

Listado de citasNueva citaUsuarios

Datos del paciente

Documento

Buscar paciente

Nombres

Apellidos

Celular

Género

Seleccionar género

Fecha de nacimiento

dd/mm/aaaa

Correo electrónico

Datos de la cita

Profesional

Selecciona una opción

Fecha

dd/mm/aaaa

Canal

WhatsApp

Observaciones

Datos de la cita

Profesional

Medicina general - Ana Gómez

Fecha

04/05/2026

Canal

WhatsApp

Observaciones

Horarios disponibles

08:00

08:30

No disponible

08:30

09:00

Disponible

09:00

09:30

Disponible

09:30

10:00

Disponible

10:00

10:30

Disponible

10:30

11:00

Disponible

11:00

11:30

Disponible

11:30

12:00

Disponible

Crear cita

Descripción del prototipo:

La pantalla de nueva cita permite al agendador registrar una cita solicitada por WhatsApp. El formulario está dividido en dos secciones principales: datos del paciente y datos de la cita. En la primera sección se capturan el número de documento, nombres, apellidos, celular, género, fecha de nacimiento opcional y correo electrónico opcional. En la segunda sección se seleccionan el médico, la fecha, el canal de contacto y la hora disponible.

El sistema debe validar que la hora seleccionada no esté ocupada. Si la franja ya fue tomada, se informa al usuario que la hora no está disponible. Si todos los datos son válidos, el sistema guarda la cita y muestra un mensaje de confirmación.

Elementos destacados:

- Búsqueda de paciente por documento con autocompletado.
- Canal por defecto en WhatsApp, reflejando el contexto real de uso.
- Horarios dinámicos que se cargan al seleccionar profesional y fecha.

Relación con la HU-2:

Este prototipo cubre el registro de datos del paciente, la selección de médico y hora, la validación de disponibilidad y la confirmación de la cita.

5.4 Prototipo H-3 — Reserva pública de cita

Reserva de citas

Agenda tu cita en línea

Primero verifica la cédula, luego completa los datos y confirma tu reserva.

Paso 1. Verifica tu cédula

Modo invitado

Estás reservando sin iniciar sesión

Primero debes ingresar tu cédula para verificar si ya existe información previa. Si no existe, podrás completar tus datos manualmente.

¿Ya tienes cuenta?

Inicia sesión para reservar más rápido

Con tu usuario podrás consultar tus citas, guardar tu perfil y evitar volver a escribir tus datos en cada reserva.

Iniciar sesión

Crear cuenta

Consultar citas

Número de cédula

Verificar cédula

Paso 2. Completa tus datos

Documento

Nombres

Apellidos

Celular

Género

Seleccionar género

Fecha de nacimiento

dd/mm/aaaa

Correo electrónico (opcional)

Paso 3. Selecciona el profesional y la fecha

Profesional: Medicina general - Ana Gómez

Fecha: dd/mm/aaaa

Ana Gómez · Medicina general

Paso 4. Elige una franja disponible

Selecciona profesional y fecha para ver las franjas disponibles.

Confirmar reserva

Descripción del prototipo:

El prototipo de agendamiento web permite que el paciente realice la reserva de una cita desde el portal público. El flujo principal está orientado a pacientes registrados y autenticados. Si el paciente no tiene cuenta, el sistema solicita el registro de usuario antes de continuar con el proceso de agendamiento.

Una vez autenticado, el paciente accede al módulo de agendamiento, visualiza la lista de médicos disponibles, selecciona un médico y consulta las franjas horarias disponibles. Finalmente, el paciente selecciona una franja y confirma la cita.

Elementos destacados:

- Modo invitado claramente diferenciado visualmente con tarjeta de color amarillo.
- Flujo lineal numerado que guía al paciente paso a paso.
- Integración con la API de disponibilidad en tiempo real.

Nota:

Aunque la interfaz contempla una opción visual de modo invitado como posible mejora de usabilidad, el flujo evaluado para la HU-3 corresponde al paciente registrado y autenticado, de acuerdo con la plantilla de historias de usuario del equipo.

Relación con la HU-3:

Este prototipo responde a los criterios de registro de usuario, visualización de médicos, visualización de horarios disponibles y confirmación de cita.

5.5 Prototipo P-4 — Configuración de disponibilidad por profesional

Disponibilidad por profesional

Profesional

Ana Gómez - Medicina general

Nombres

Ana

Apellidos

Gómez

Especialidad

Medicina General

Día

Lunes

Inicio

08:00 a. m.

Fin

12:00 p. m.

Intervalo

30

☒ Activo

Eliminar

Día

Miércoles

Inicio

02:00 p. m.

Fin

06:00 p. m.

Intervalo

30

☒ Activo

Eliminar

Agregar franja

Eliminar profesional

Guardar disponibilidad

Descripción del prototipo:

La pantalla de configuración permite al administrador definir los parámetros necesarios para que el sistema genere correctamente la disponibilidad de citas. Desde esta interfaz se puede configurar la ventana de agendamiento, es decir, el número de semanas disponibles para reservar citas. También se permite seleccionar un médico, definir los días de atención, configurar la hora de inicio y fin de cada franja horaria, y establecer el intervalo en minutos entre citas.

Estos parámetros son utilizados por el sistema para calcular las franjas disponibles que posteriormente se muestran al paciente y al agendador.

Elementos destacados:

- Múltiples franjas por día soportadas (lunes 8-12, miércoles 14-18).
- Checkbox de activación por franja sin necesidad de eliminarla.
- Confirmación visual con botón de guardado prominente.

Relación con la HU-4:

Este prototipo cubre la configuración de la ventana de agendamiento, los días de atención, las franjas horarias y el intervalo entre citas.

5.6 Prototipo H-5 — Exportación de citas

☐ Seleccionar varias fechas

Profesional

Medicina general - Ana Gómez

Fecha

04/05/2026

Buscar

Descargar PDF

CSV

Excel

A1

⌵

:

✕

✓

f_x

⌵

Listado de citas - Ana Gómez - Medicina general - 2026-05-04

	A	B	C	D	E	F	G	H	I	J	K
1	Listado de citas - Ana Gómez - Medicina general - 2026-05-04										
2	Fecha	Hora inicio	Hora fin	Paciente	Documento	Celular	Profesional	Especialidad	Estado	Canal	
3	4/05/2026	8:00	8:30	Paciente Demo	1000000001	3001234567	Ana Gómez	Medicina general	Programada	Web	
4											
5											

Descripción del prototipo:

La funcionalidad de exportación está integrada en la pantalla de listado de citas. El usuario autorizado selecciona un médico y una fecha específica, y luego ejecuta la opción de exportar. El sistema genera un archivo en formato CSV con las citas correspondientes al médico y la fecha seleccionados.

El archivo CSV contiene las columnas requeridas para la operación diaria: hora, nombre del paciente, documento, celular, especialidad, observaciones y estado. Este archivo puede abrirse en una hoja de cálculo, editarse e imprimirse para apoyar la organización de los pacientes durante la jornada de atención.

En caso de que no existan citas para el médico y la fecha seleccionados, el sistema informa que no hay citas para exportar y no genera el archivo. Además, el acceso a esta funcionalidad está restringido a médicos, terapeutas y agendadores; si un paciente intenta acceder a la exportación, el sistema debe denegar la solicitud con error 403.

Elementos destacados:

- Tres formatos de exportación disponibles: PDF, CSV y Excel.
- Nombre del archivo generado automáticamente con los parámetros de búsqueda.
- Encabezado con título descriptivo en la primera fila del archivo.

Relación con la HU-5:

Este prototipo responde a los criterios de selección de parámetros, generación del archivo CSV, contenido del archivo, manejo del caso sin citas y control de acceso por rol.

Test de Evaluación de Usabilidad

Metodología: Evaluación heurística basada en los 10 principios de Jakob Nielsen. Escala de cumplimiento: ✓ Cumple / ~ Parcial / ✗ No cumple.

HU-1 — Listado de citas

N°	Criterio	Descripción	Cumplimiento	Observaciones
1	Visibilidad del estado	La tabla muestra el estado actual de cada cita con texto claro	✓ Cumple	El estado es legible y actualizable desde la misma fila
2	Coincidencia con el mundo real	Términos como "Profesional", "Fecha", "Buscar" son familiares	✓ Cumple	Lenguaje natural y cercano al usuario
3	Control y libertad	El usuario puede cambiar filtros libremente en cualquier momento	✓ Cumple	No hay flujos bloqueantes
4	Consistencia	Mismo estilo visual en todos los controles del portal	✓ Cumple	Botones, dropdowns y campos consistentes
5	Prevención de errores	Restricción de cambio de estado según hora	✓ Cumple	Mensaje informativo visible bajo el dropdown
6	Reconocimiento antes que recuerdo	Dropdowns con valores precargados	✓ Cumple	El usuario no necesita recordar nombres de médicos
7	Flexibilidad	Múltiples formatos de exportación disponibles	✓ Cumple	PDF, CSV y Excel en la misma pantalla
8	Diseño estético	Interfaz limpia con buena jerarquía visual	✓ Cumple	Sin elementos innecesarios
9	Recuperación de errores	Mensaje cuando no hay citas para los filtros seleccionados	✓ Cumple	Feedback claro al usuario
10	Ayuda y documentación	Sin tooltips ni ayuda contextual	~ Parcial	Recomendable agregar tooltips en botones de exportación

HU-2 — Nueva cita (agendador)

N°	Criterio	Descripción	Cumplimiento	Observaciones
----	----------	-------------	--------------	---------------

1	Visibilidad del estado	Secciones claramente diferenciadas con títulos	✓ Cumple	"Datos del paciente" y "Datos de la cita" bien separados
2	Coincidencia con el mundo real	Campos reflejan el formulario real de registro	✓ Cumple	Género, cédula, celular son términos conocidos
3	Control y libertad	El agendador puede editar cualquier campo libremente	✓ Cumple	Sin restricciones de flujo
4	Prevención de errores	Búsqueda previa de paciente evita duplicados	✓ Cumple	Autocompletado reduce errores de digitación
5	Reconocimiento antes que recuerdo	Horarios disponibles se cargan automáticamente	✓ Cumple	El agendador no necesita consultar una agenda externa
6	Flexibilidad	Canal por defecto en WhatsApp es configurable	✓ Cumple	Refleja el canal más usado en el contexto real
7	Diseño estético	Formulario limpio en dos columnas	✓ Cumple	Buena distribución del espacio
8	Consistencia	Mismo patrón de campos que el portal público	✓ Cumple	Experiencia coherente entre portales
9	Recuperación de errores	Validación de campos obligatorios	~ Parcial	Falta mensaje de error inline por campo
10	Ayuda y documentación	Sin indicación de campos opcionales vs obligatorios	~ Parcial	Recomendable marcar campos requeridos con asterisco

HU-3 — Reserva pública de cita

N°	Criterio	Descripción	Cumplimiento	Observaciones
1	Visibilidad del estado	Pasos numerados muestran el progreso del flujo	✓ Cumple	El paciente sabe en qué paso está en todo momento
2	Coincidencia con el mundo real	Lenguaje orientado al paciente, sin tecnicismos	✓ Cumple	"Agenda tu cita en línea" es directo y claro
3	Control y libertad	Modo invitado y modo con cuenta claramente diferenciados	✓ Cumple	El paciente elige su camino sin confusión

4	Prevención de errores	Verificación de cédula antes de completar datos	✓ Cumple	Evita registros duplicados
5	Reconocimiento antes que recuerdo	Franjas disponibles se muestran dinámicamente	✓ Cumple	El paciente solo ve opciones reales disponibles
6	Flexibilidad	Acceso como invitado sin necesidad de registro	✓ Cumple	Reduce fricción para nuevos pacientes
7	Diseño estético	Tarjeta amarilla de modo invitado destaca sin saturar	✓ Cumple	Buen uso del color para diferenciar estados
8	Consistencia	Mismo estilo visual que el resto del sistema	✓ Cumple	Tipografía y colores consistentes
9	Recuperación de errores	Mensaje si la cédula no existe	✓ Cumple	Redirige al registro de nuevo paciente
10	Ayuda y documentación	Texto explicativo en modo invitado	✓ Cumple	Explica claramente las limitaciones del modo invitado

HU-4 — Configuración de disponibilidad

N°	Criterio	Descripción	Cumplimiento	Observaciones
1	Visibilidad del estado	Checkbox "Activo" por franja muestra estado claramente	✓ Cumple	El administrador ve de inmediato qué franjas están activas
2	Control y libertad	Agregar y eliminar franjas en cualquier momento	✓ Cumple	Sin restricciones de flujo
3	Prevención de errores	Botón "Eliminar profesional" diferenciado en rojo	✓ Cumple	Color advierte sobre acción destructiva
4	Reconocimiento antes que recuerdo	Dropdown con días de la semana en español	✓ Cumple	No requiere recordar formato de días
5	Flexibilidad	Múltiples franjas por día soportadas	✓ Cumple	Lunes mañana y miércoles tarde configurados simultáneamente
6	Diseño estético	Formulario estructurado en filas por franja	✓ Cumple	Fácil de escanear visualmente

7	Consistencia		Mismos controles de tiempo en todas las franjas	✓ Cumple	Patrón repetible y predecible
8	Recuperación de errores		Sin validación visible de solapamiento de franjas	~ Parcial	Recomendable validar que las franjas no se superpongan
9	Ayuda		Sin confirmación antes de eliminar una franja	~ Parcial	Recomendable agregar diálogo de confirmación
10	Accesibilidad		Controles de hora con selector nativo del navegador	✓ Cumple	Compatible con teclado y accesibilidad básica

HU-5 — Exportación de citas

N°	Criterio		Descripción	Cumplimiento	Observaciones
1	Visibilidad del estado		Botones de exportación visibles sin necesidad de buscarlos	✓ Cumple	PDF, CSV y Excel en la misma línea de filtros
2	Coincidencia con el mundo real		Formatos estándar reconocidos por los usuarios	✓ Cumple	PDF, CSV y Excel son formatos universales
3	Control y libertad		El usuario elige el formato de exportación	✓ Cumple	Tres opciones disponibles simultáneamente
4	Prevención de errores		Exportación solo disponible cuando hay resultados	✓ Cumple	Evita generar archivos vacíos
5	Reconocimiento		Nombre del archivo generado incluye médico y fecha	✓ Cumple	El usuario identifica el archivo sin abrirlo
6	Flexibilidad		Soporta exportación de múltiples fechas	✓ Cumple	Checkbox "Seleccionar varias fechas" disponible
7	Diseño estético		Botones de exportación alineados y diferenciados	✓ Cumple	Jerarquía visual clara entre Buscar y exportar
8	Consistencia		Mismo estilo de botones en todo el portal	✓ Cumple	Coherente con el resto de la interfaz
9	Recuperación de errores		Sin indicador de progreso durante la generación del archivo	~ Parcial	Recomendable agregar spinner en archivos grandes

10	Ayuda	Sin tooltip que explique diferencia entre CSV y Excel	~ Parcial	Recomendable para usuarios no técnicos
----	-------	---	-----------	--

Resumen General de Usabilidad

Historia de Usuario	Criterios evaluados	Cumplidos	Parciales	Resultado
HU-1 — Listado de citas	10	9	1	9/10 — Muy bueno
HU-2 — Nueva cita agendador	10	8	2	8/10 — Bueno
HU-3 — Reserva pública	10	10	0	10/10 — Excelente
HU-4 — Configuración disponibilidad	10	8	2	8/10 — Bueno
HU-5 — Exportación CSV/Excel	10	8	2	8/10 — Bueno

Conclusión: El sistema Piedrazul presenta un nivel de usabilidad general alto. El flujo de reserva pública (HU-3) obtuvo la calificación más alta por su diseño guiado y centrado en el paciente. Las áreas de mejora identificadas son menores y corresponden principalmente a la ausencia de tooltips, mensajes de error inline y diálogos de confirmación en acciones destructivas, aspectos que pueden incorporarse en iteraciones futuras sin afectar la funcionalidad core del sistema.

6. Planificación del Sprint 2

La planificación del Sprint 2 se realizó utilizando un tablero de gestión de tareas en Trello/Jira, con el objetivo de organizar el trabajo del equipo, distribuir responsabilidades y hacer seguimiento al avance de las funcionalidades priorizadas para la segunda iteración del sistema Piedrazul.

El Sprint 2 se enfocó en implementar y documentar las historias de usuario HU-1 a HU-5, correspondientes al listado de citas, creación manual de citas, agendamiento autónomo desde la web, configuración de parámetros de disponibilidad y exportación de citas en formato CSV. Adicionalmente, se incluyeron tareas técnicas relacionadas con autenticación y autorización mediante JWT y Keycloak, pruebas de seguridad con OWASP ZAP, documentación arquitectónica y validación de prototipos.

6.1 Objetivo del Sprint 2

El objetivo principal del Sprint 2 fue consolidar una versión funcional del sistema Piedrazul que permitiera gestionar el proceso de agendamiento de citas desde dos perspectivas: el portal interno utilizado por agendadores, médicos y administradores, y el portal público utilizado por pacientes.

Desde el punto de vista funcional, el sprint buscó entregar las funcionalidades de mayor valor para el cliente en esta iteración. Desde el punto de vista arquitectónico, el sprint permitió validar decisiones como el uso de una aplicación web SPA, una API REST desacoplada, persistencia en PostgreSQL, autenticación con Keycloak y separación de responsabilidades mediante Clean Architecture.

6.2 Historias de usuario incluidas en el Sprint

Historia	Descripción	Resultado esperado
HU-1	Listar citas médicas por médico y fecha	Visualizar las citas programadas y el total de citas del día.
HU-2	Crear una nueva cita desde el portal interno	Registrar una cita solicitada por WhatsApp, validando datos del paciente y disponibilidad.
HU-3	Agendar una cita desde la web	Permitir que un paciente registrado y autenticado seleccione médico, horario y confirme la cita.
HU-4	Configurar parámetros del sistema de agendamiento	Definir ventana de agendamiento, días de atención, franjas horarias e intervalos por médico.
HU-5	Exportar citas en formato CSV	Generar un archivo compatible con hojas de cálculo para apoyar la jornada de atención.

6.3 Organización del tablero de trabajo

El tablero de Sprint 2 se organizó en columnas que representan el flujo de trabajo del equipo. Esta estructura permitió visualizar el estado de cada actividad y controlar el avance de las tareas durante la iteración.

Columna del tablero	Propósito
Backlog	Contiene las tareas identificadas y pendientes de priorización.
Lista de tareas por hacer	Incluye las tareas seleccionadas para el Sprint 2.
En proceso	Agrupar las actividades que se encuentran en desarrollo o documentación.
Revisión	Contiene tareas terminadas que requieren validación, pruebas o revisión por parte del equipo.
Hecho	Agrupar las tareas completadas y aceptadas dentro del sprint.

6.4 Tipos de tareas planificadas

Durante el Sprint 2, las tareas se agruparon en cuatro categorías principales:

Categoría	Descripción
Funcionalidad	Implementación de las historias de usuario HU-1 a HU-5.
Interfaz de usuario	Construcción de prototipos funcionales y validación de usabilidad.
Seguridad	Configuración de autenticación, autorización por roles y pruebas con OWASP ZAP.

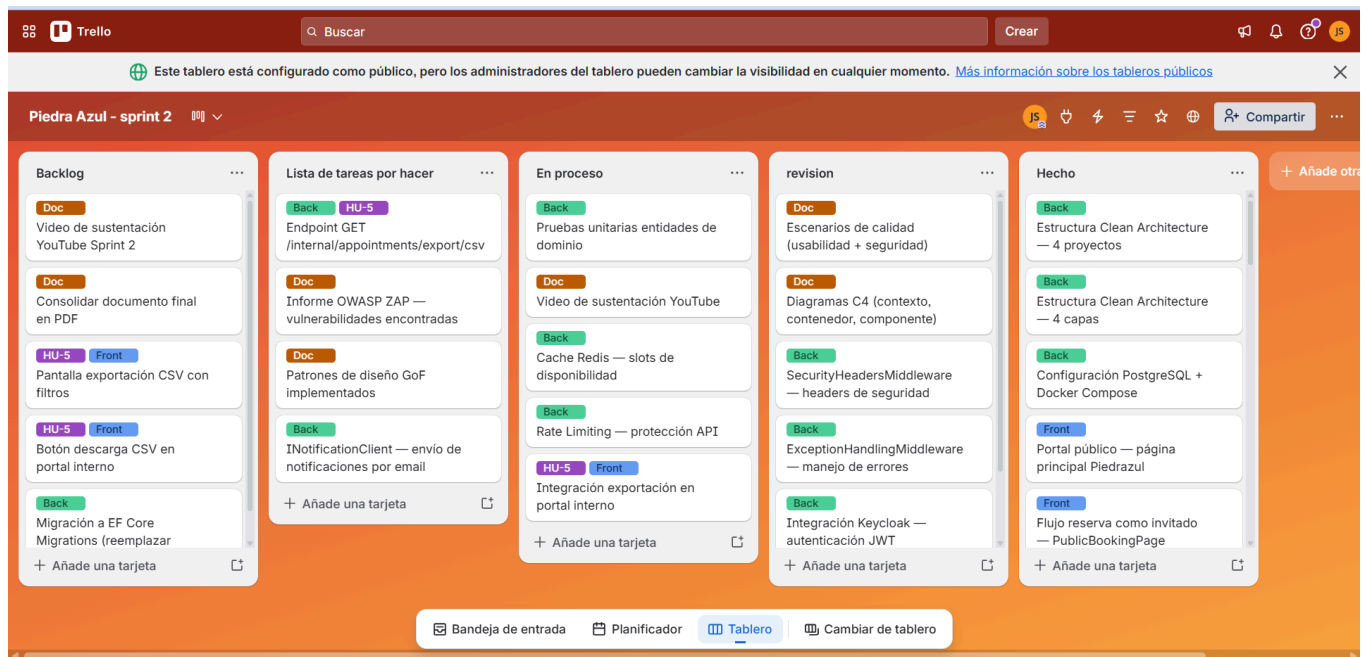
Arquitectura y documentación	Elaboración de diagramas C4, vistas 4+1, descripción de patrones GoF y actualización del SAD.
------------------------------	---

6.5 Evidencia de planificación

La siguiente figura presenta el tablero de Trello utilizado para la planificación y seguimiento del Sprint 2. El tablero está organizado en columnas de flujo de trabajo: **Backlog**, **Lista de tareas por hacer**, **En proceso**, **Revisión** y **Hecho**. Esta organización permitió visualizar el estado de avance de las actividades funcionales, técnicas, de documentación, seguridad y arquitectura durante la iteración.

En el tablero se evidencian tareas asociadas a la exportación de citas en formato CSV, integración con Keycloak para autenticación JWT, protección de la API mediante Rate Limiting, uso de Redis para disponibilidad de citas, pruebas unitarias de entidades de dominio, pruebas de seguridad con OWASP ZAP, elaboración de diagramas C4, documentación de patrones GoF, configuración de PostgreSQL con Docker Compose y consolidación del documento final en PDF.

Figura . Pantallazo de planificación de tareas del Sprint 2 en Trello/Jira.



6.6 Relación entre planificación y arquitectura

La planificación del Sprint 2 incluyó actividades funcionales y técnicas que impactan directamente la arquitectura del sistema. Las tareas de exportación CSV y consulta de citas validan los servicios de aplicación relacionados con la gestión de citas. Las tareas de integración con Keycloak, SecurityHeadersMiddleware, ExceptionHandlingMiddleware y Rate Limiting fortalecen los mecanismos de seguridad, autenticación, autorización y manejo controlado de errores.

Asimismo, las tareas relacionadas con Redis, PostgreSQL, Docker Compose y Clean Architecture evidencian decisiones orientadas a mejorar la modificabilidad, portabilidad y mantenibilidad del sistema. La inclusión de pruebas unitarias, OWASP ZAP, diagramas C4 y patrones GoF demuestra que la planificación del sprint no se limitó al desarrollo funcional, sino que también incorporó actividades de validación técnica y documentación arquitectónica.

7. Escenarios de calidad del atributo de seguridad

Esta sección presenta los escenarios de calidad asociados al atributo de seguridad para el sistema Piedrazul. Los escenarios fueron definidos a partir de los requisitos no funcionales del segundo corte, especialmente la autenticación y autorización basada en tokens JWT mediante Keycloak, y la validación de vulnerabilidades mediante OWASP ZAP.

El atributo de seguridad tiene una prioridad arquitectónica alta debido a que el sistema gestiona información sensible de pacientes, citas médicas y datos clínicos básicos. Además, el sistema contempla diferentes roles de usuario, como paciente, agendador, médico, terapeuta y administrador, por lo cual es necesario restringir el acceso a las funcionalidades según los permisos de cada actor.

Los escenarios seleccionados permiten validar dos riesgos relevantes:

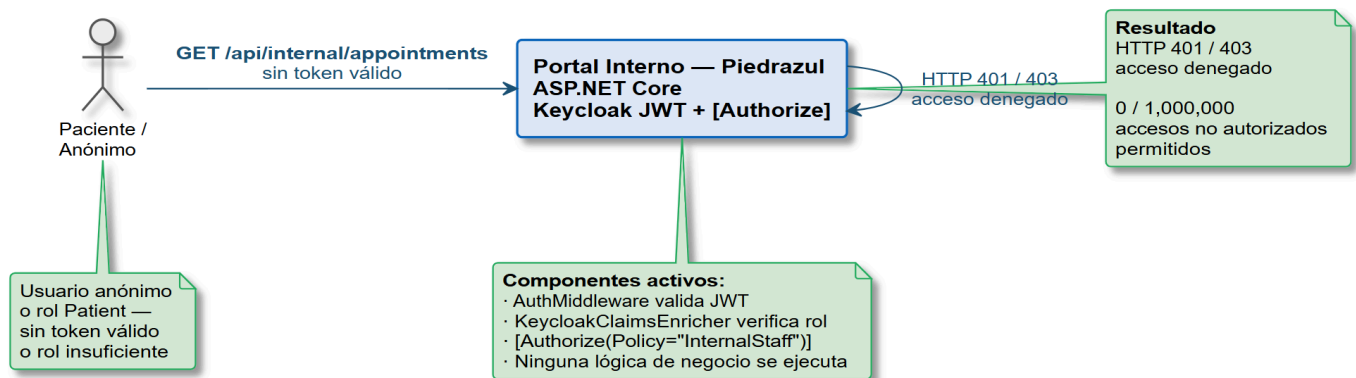
- Acceso no autorizado a funcionalidades internas del sistema.
- Abuso de endpoints públicos mediante múltiples peticiones automatizadas.

7.1 Resumen de escenarios de calidad

ID	Escenario	Atributo de calidad	Riesgo mitigado	Prioridad
QS1	Acceso no autorizado al portal interno	Seguridad	Acceso indebido a citas, pacientes o funcionalidades administrativas	Alta
QS2	Protección contra ataques de fuerza bruta y abuso de API	Seguridad	Uso automatizado o malicioso de endpoints públicos	Alta

7.2 QS1 Acceso no autorizado al portal interno

Figura 1. Escenario de Seguridad QS1
Acceso no autorizado al portal interno



ID: QS1

Nombre: Seguridad : Acceso no autorizado a endpoints del portal interno

Sinopsis: El sistema debe garantizar que solo los usuarios autenticados y con el rol correcto puedan acceder a los recursos protegidos de la API. Esto es crítico para evitar que un paciente o usuario anónimo pueda consultar, crear o exportar citas del portal interno.

Entorno: El sistema Piedrazul está en funcionamiento normal. Los endpoints del portal interno (/api/internal/*) requieren autenticación JWT y rol Admin, Scheduler o Doctor. El sistema usa Keycloak como proveedor de identidad. El Middleware de Autenticación está activo en modo Keycloak, validando tokens contra las claves públicas JWKS del servidor de identidad.

Cambio en el entorno: Un usuario anónimo o con rol Patient intenta acceder directamente a un endpoint protegido del portal interno, por ejemplo GET /api/internal/appointments, sin token válido o con token de paciente.

Comportamiento esperado: El sistema rechaza la petición antes de ejecutar cualquier lógica de negocio. Si no hay token, retorna HTTP 401 Unauthorized. Si hay token pero el rol no es suficiente, retorna HTTP 403 Forbidden. En ningún caso se expone información de citas ni de pacientes.

Medida: El 100% de las peticiones no autorizadas deben ser rechazadas con el código HTTP correcto (401 o 403). Cero peticiones deben llegar a la capa de servicio sin autenticación válida.

Prioridad Arquitectónica: Alta

Aplicación: Portal interno API REST ASP.NET Core con Keycloak JWT

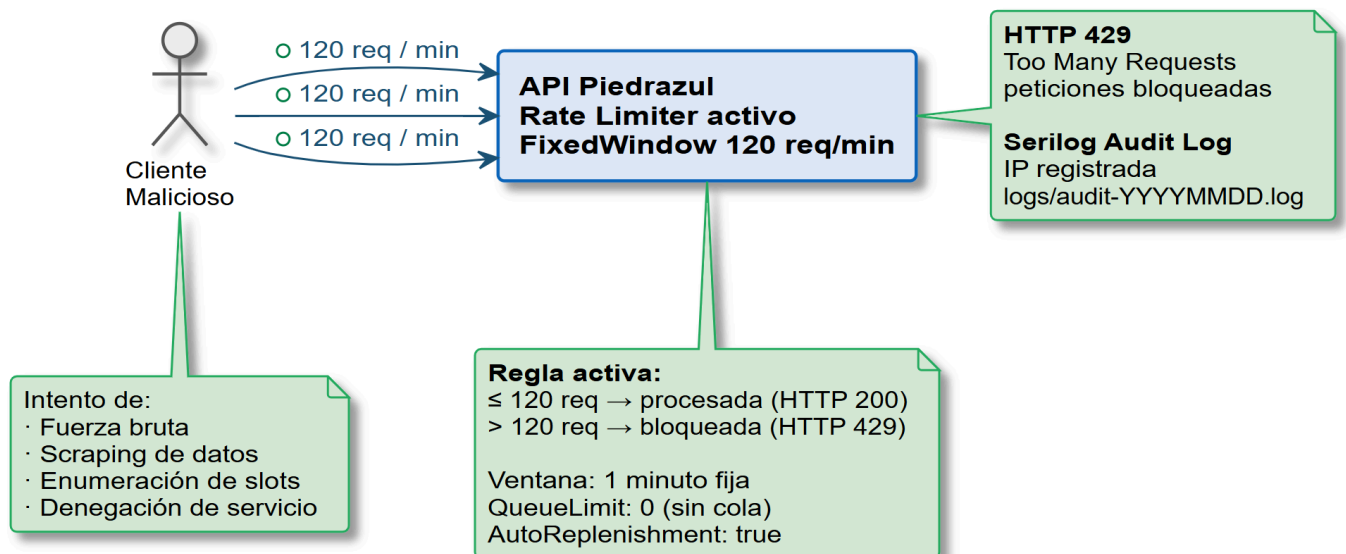
Táctica arquitectónica: *Autenticar y autorizar actores* implementada mediante JWT Bearer Authentication y políticas de autorización declarativas ([Authorize(Policy)]) en los controllers. El KeycloakClaimsEnricher mapea los roles del realm de Keycloak a ClaimTypes.Role para que las políticas funcionen correctamente.

Componentes involucrados:

- Middleware de Autenticación (Piedrazul.Api)
- KeycloakClaimsEnricher (Piedrazul.Infrastructure)
- InternalController, AdminController con [Authorize]
- Servidor de Identidad Keycloak 26

7.3 QS2 Protección contra ataques de fuerza bruta y abuso de API

Figura 2. Escenario de Seguridad QS2
Fuerza bruta bloqueada por Rate Limiting



ID: QS2

Nombre: Seguridad : Protección contra ataques de fuerza bruta y abuso de la API

Sinopsis: El sistema debe limitar la cantidad de peticiones que un mismo cliente puede realizar en un periodo de tiempo, para proteger los endpoints públicos y de autenticación contra ataques de fuerza bruta, scraping masivo y denegación de servicio.

Entorno: El sistema Piedrazul está en funcionamiento normal y accesible desde internet. Los endpoints públicos (/api/public/*) no requieren autenticación y son accesibles por cualquier usuario. El Rate Limiter está configurado con FixedWindowLimiter en 120 peticiones por minuto por cliente (basado en IP).

Cambio en el entorno: Un cliente malicioso intenta enviar más de 120 peticiones en menos de un minuto al mismo endpoint, ya sea para adivinar datos de pacientes, saturar el servidor o enumerar slots de disponibilidad masivamente.

Comportamiento esperado: El sistema detecta que el cliente superó el límite configurado (120 peticiones por minuto) y responde con HTTP 429 Too Many Requests para todas las peticiones excedentes hasta que la ventana de tiempo se renueve. Las peticiones dentro del límite se procesan normalmente sin degradación de rendimiento. Cada petición rechazada queda registrada en los logs de auditoría generados por Serilog en logs/audit-YYYYMMDD.log.

Medida: Ningún cliente debe poder superar 120 peticiones por minuto. El tiempo de respuesta para las peticiones dentro del límite no debe superar los 300ms en condiciones normales. Las peticiones rechazadas deben registrarse en los logs de auditoría con la IP del cliente.

Prioridad Arquitectónica: Alta

Aplicación: API REST ASP.NET Core FixedWindowLimiter en Program.cs

Táctica arquitectónica: *Limitar exposición* — el Rate Limiter actúa como primera línea de defensa ante abuso de los endpoints públicos, complementando la autenticación JWT del QS1. Cubre la categoría OWASP A07:2021.

Componentes involucrados:

- RateLimiter (ASP.NET Core FixedWindowLimiter)
- ExceptionHandlingMiddleware (retorna 429 limpio)
- Serilog logs de auditoría en logs/audit-YYYYMMDD.log

7.4 Relación con OWASP Top 10

Los escenarios definidos se relacionan con categorías relevantes de OWASP Top 10:

Escenario	Categoría OWASP relacionada	Justificación
QS1	A01: Broken Access Control	Previene que usuarios sin permisos accedan a funcionalidades internas.
QS1	A07: Identification and Authentication Failures	Valida autenticación segura mediante tokens JWT.
QS2	A07: Identification and Authentication Failures	Reduce abuso de endpoints relacionados con autenticación o acceso público.
QS2	A05: Security Misconfiguration	Refuerza la configuración segura del backend mediante límites y respuestas controladas.

7.5 Evidencia de validación

La validación de los escenarios de seguridad se complementó mediante pruebas con OWASP ZAP, herramienta utilizada para identificar posibles vulnerabilidades en aplicaciones web. Para esta iteración se analizaron al menos dos aspectos relacionados con OWASP Top 10, priorizando control de acceso, autenticación, exposición de endpoints y comportamiento del sistema ante peticiones no autorizadas o abusivas.

Los resultados de estas pruebas se documentan en el informe breve de vulnerabilidades generado para el Sprint 2.

8. Arquitectura y diseño de software usando el modelo C4 y 4+1 vistas.

La arquitectura del sistema Piedrazul está representada mediante dos frameworks complementarios: el **modelo C4** de Simon Brown, que describe la arquitectura desde el nivel de contexto hasta el nivel de componentes de forma progresiva; y el framework **4+1 vistas** de Philippe Kruchten, que complementa la visión estática con perspectivas de comportamiento, desarrollo y despliegue.

La arquitectura implementada sigue el patrón de **Clean Architecture** con separación en cuatro capas: Domain, Application, Infrastructure y Presentation (Api). Esta decisión permite que la lógica de negocio sea independiente de los frameworks de infraestructura, facilitando las pruebas unitarias y la modificabilidad del sistema.

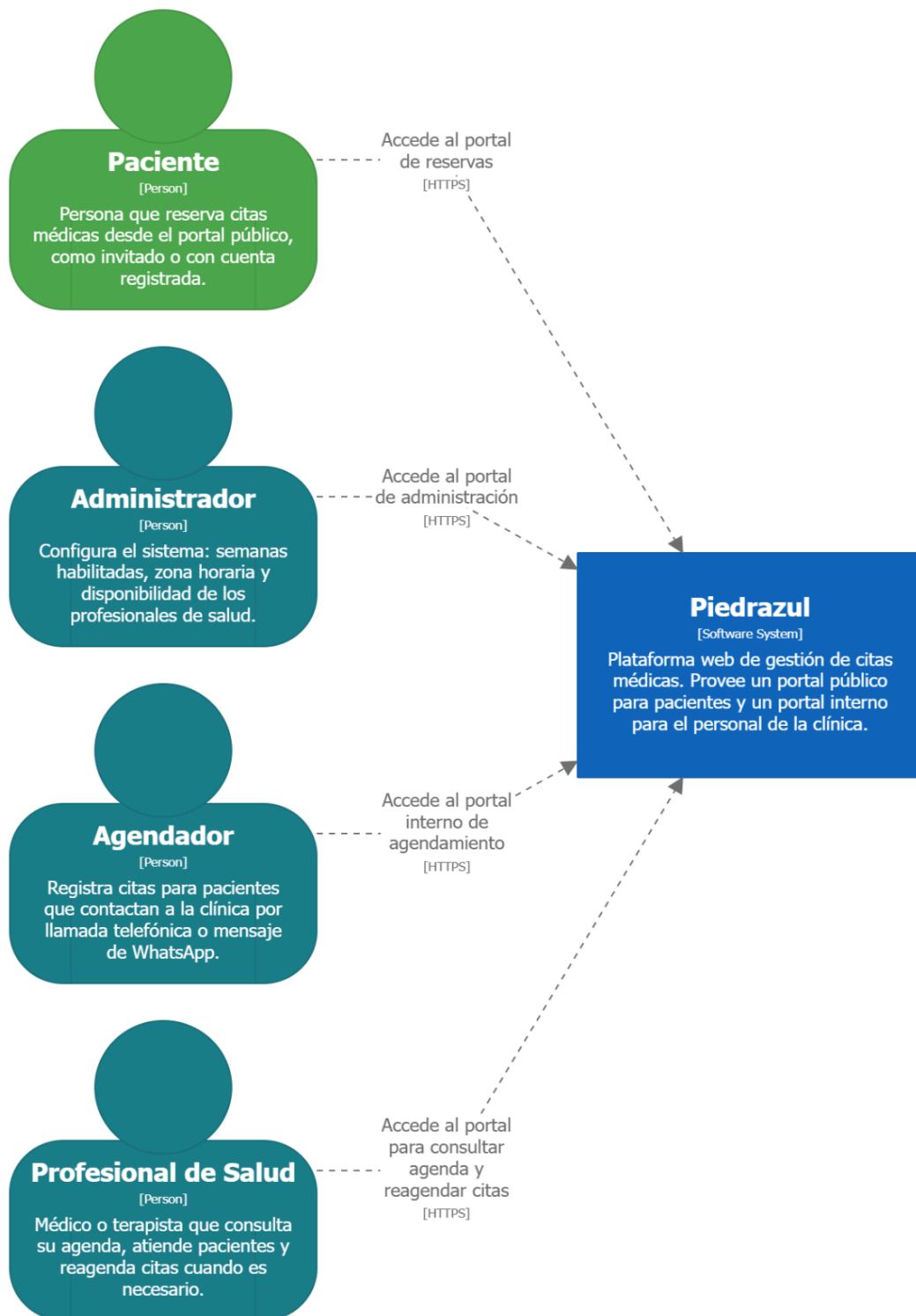
Los diagramas fueron modelados con **Structurizr DSL** (C4) y **PlantUML** (UML), herramientas estándar de la industria para documentación arquitectónica.

8.1 Modelo C4

El modelo C4 describe el sistema Piedrazul en cuatro niveles de abstracción progresiva: contexto, contenedores, componentes y despliegue. Adicionalmente se incluyen dos diagramas dinámicos que representan los flujos de interacción más relevantes del sistema.

Código Structurizr [C_piedrazul-c4-final.dsl](#)

8.1.1 Diagrama de Sistema (*System Landscape*)



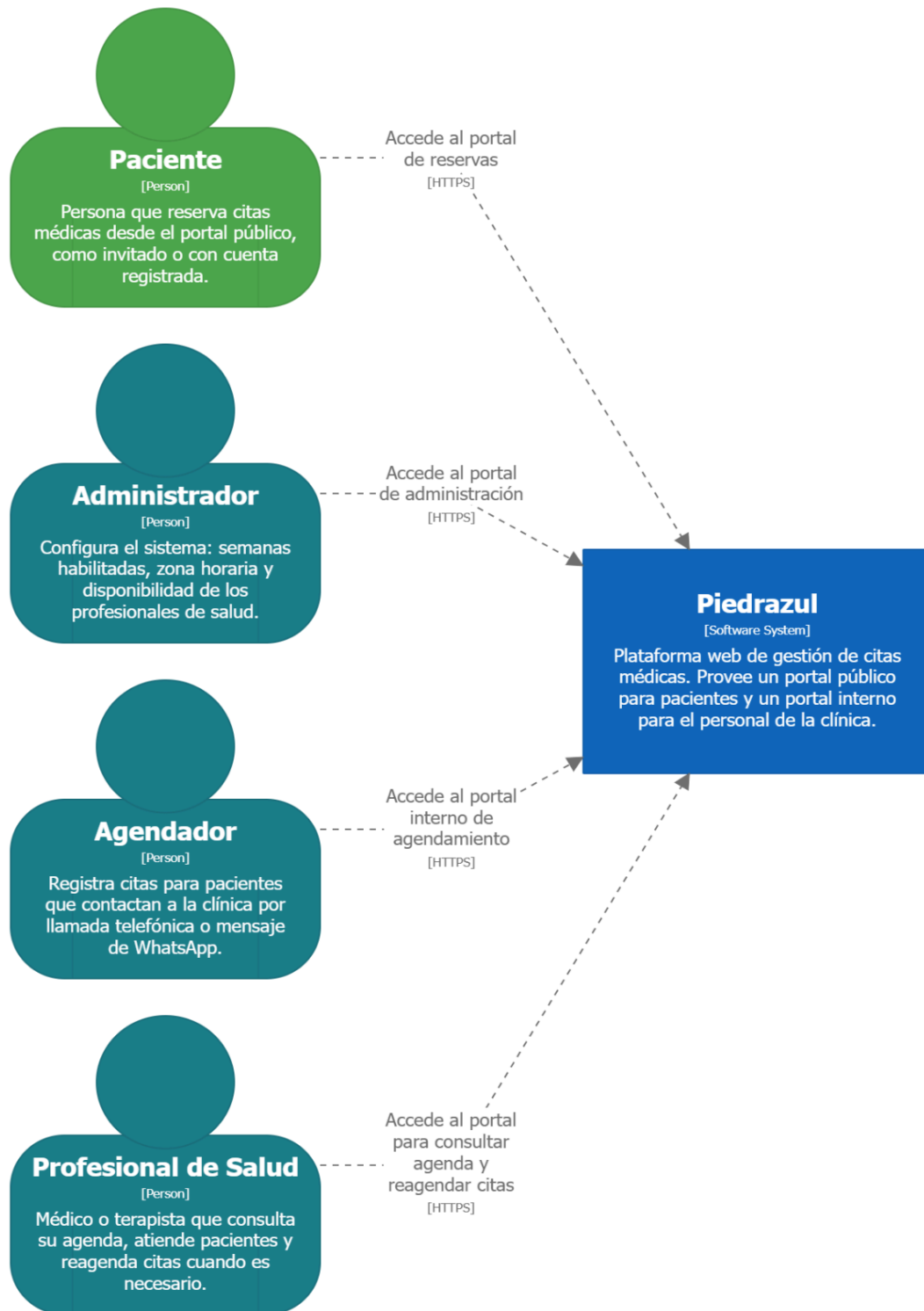
System Landscape View

Ecosistema del sistema Piedrazul: actores y sistema principal.

domingo, 3 de mayo de 2026, 8:41 a. m. hora estándar de Colombia

El diagrama de sistema presenta el ecosistema completo de Piedrazul: los cuatro actores que interactúan con el sistema (Paciente, Administrador, Agendador y Profesional de Salud) y el sistema principal. El **Paciente** es el único actor externo a la organización, representado en verde siguiendo la notación oficial de Structurizr. Los actores internos (personal de la clínica) se representan en azul teal.

8.1.2 Diagrama de Contexto (System Context — Nivel 1)



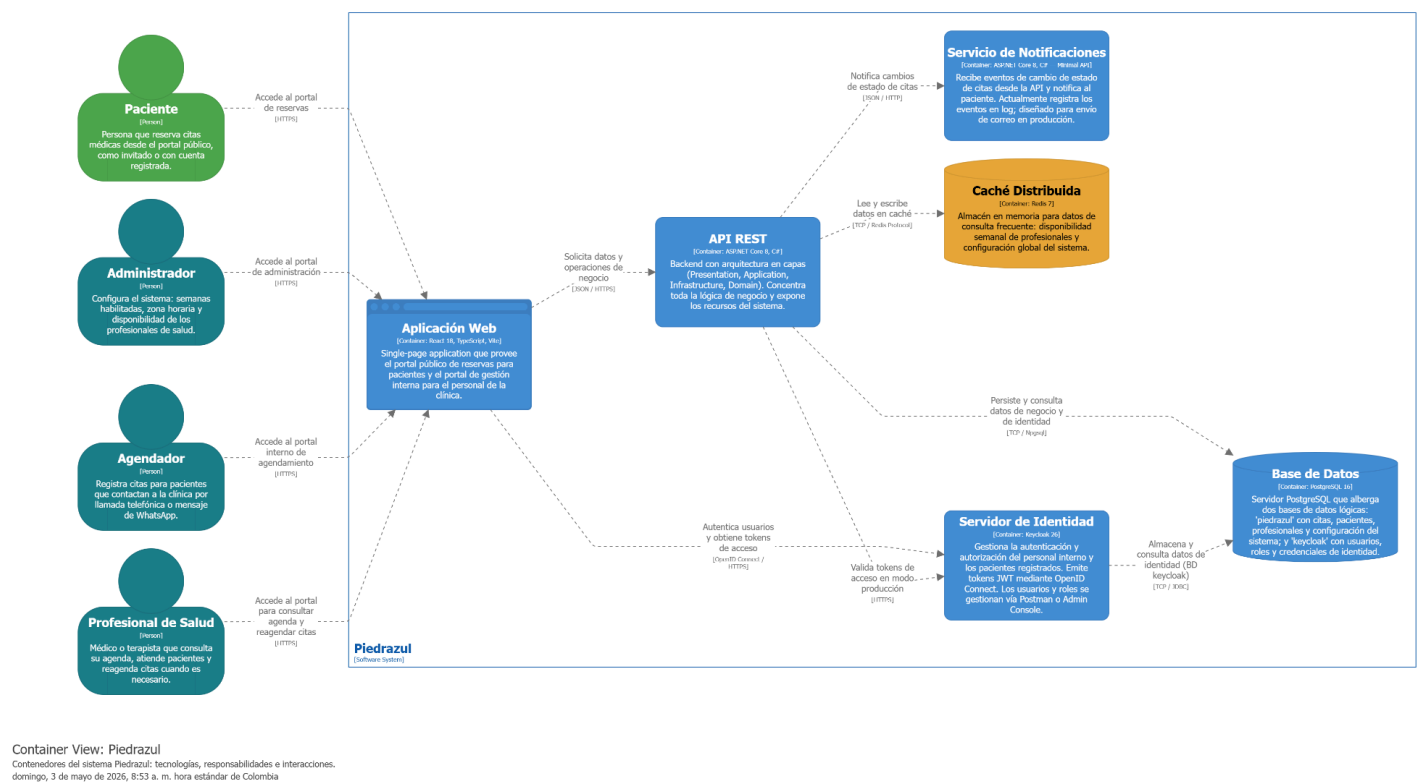
System Context View: Piedrazul

El sistema Piedrazul en contexto: personas que interactúan con él.
domingo, 3 de mayo de 2026, 8:51 a. m. hora estándar de Colombia

El diagrama de contexto sitúa el sistema Piedrazul dentro de su entorno. Muestra las cuatro personas que interactúan con él y sus relaciones de alto nivel:

- El **Paciente** reserva citas y gestiona su perfil desde el portal público.
- El **Agendador** registra citas del portal interno para pacientes que contactan por llamada o WhatsApp.
- El **Profesional de Salud** consulta su agenda de citas del día.
- El **Administrador** configura la disponibilidad y los parámetros del sistema.

8.1.3 Diagrama de Contenedores (Nivel 2)



El diagrama de contenedores descompone el sistema en sus piezas ejecutables e interconectadas:

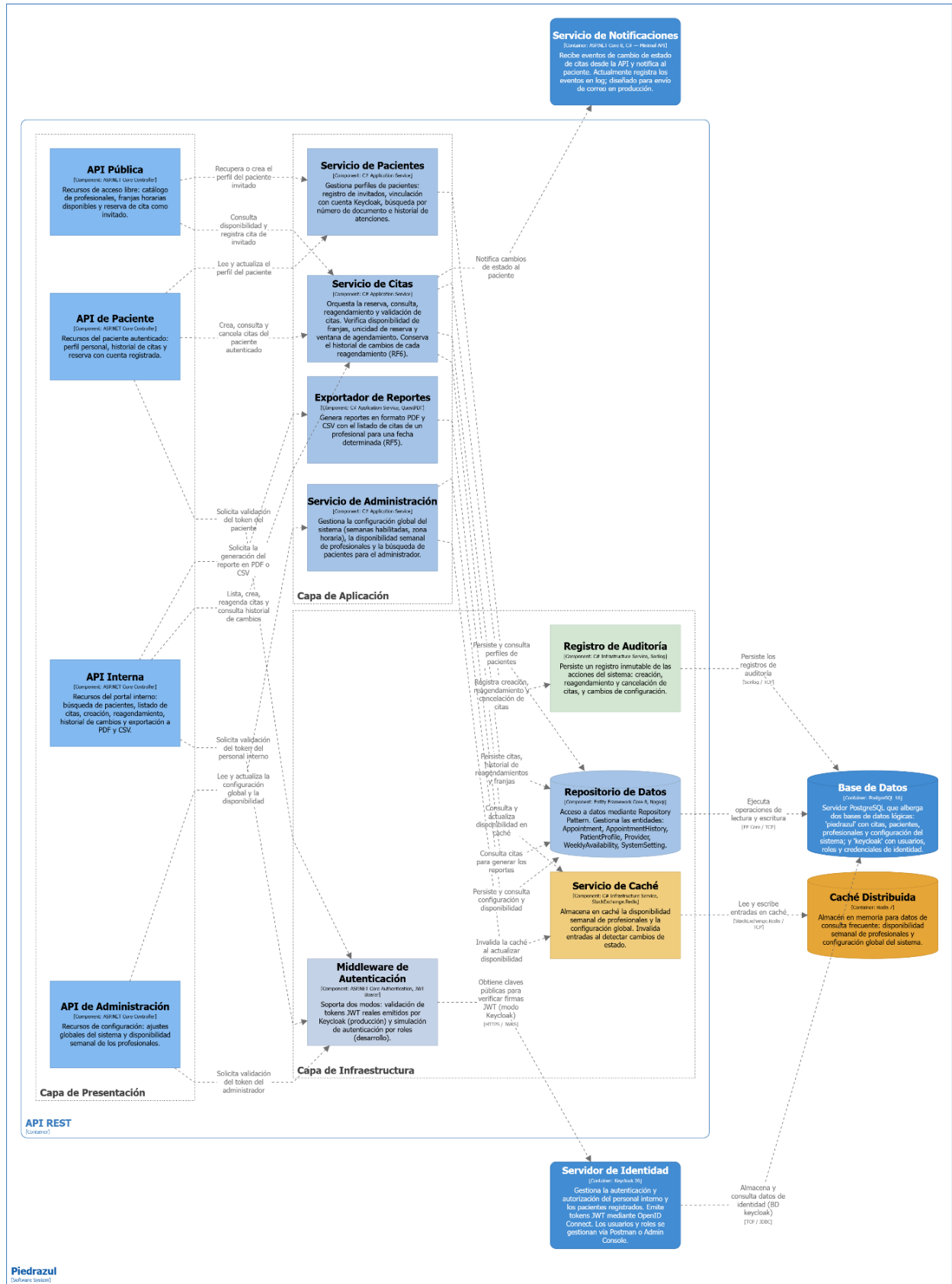
Contenedor	Tecnología	Responsabilidad
Aplicación Web	React 18, TypeScript, Vite	Portal público y portal interno
API REST	ASP.NET Core 8, C#	Lógica de negocio y endpoints
Servicio de Notificaciones	ASP.NET Core 8, Minimal API	Notificaciones de cambio de estado
Caché Distribuida	Redis 7	Disponibilidad semanal en memoria

Base de Datos	PostgreSQL 16	Citas, pacientes, configuración e identidad
Servidor de Identidad	Keycloak 26	Autenticación y autorización JWT

La **Base de Datos** alberga dos esquemas lógicos en el mismo servidor PostgreSQL: `piedrazu1` para los datos de negocio y `keycloak` para los datos de identidad, creados automáticamente por el script de inicialización de Docker.

Nota: Dado que la imagen es un poco más grande, representa ilegibilidad; colocando ruta de imagen.

[Containers.png](#)



El diagrama de componentes revela la estructura interna de la API REST, organizada en tres capas arquitectónicas claramente delimitadas:

Capa de Presentación — recibe peticiones HTTP y delega a los servicios:

- *API Pública*: catálogo de profesionales y reserva sin autenticación.
- *API de Paciente*: perfil e historial para el paciente autenticado.
- *API Interna*: gestión de citas del portal interno (rol Scheduler / Doctor).
- *API de Administración*: configuración del sistema (rol Admin).

Capa de Aplicación — orquesta los casos de uso del dominio:

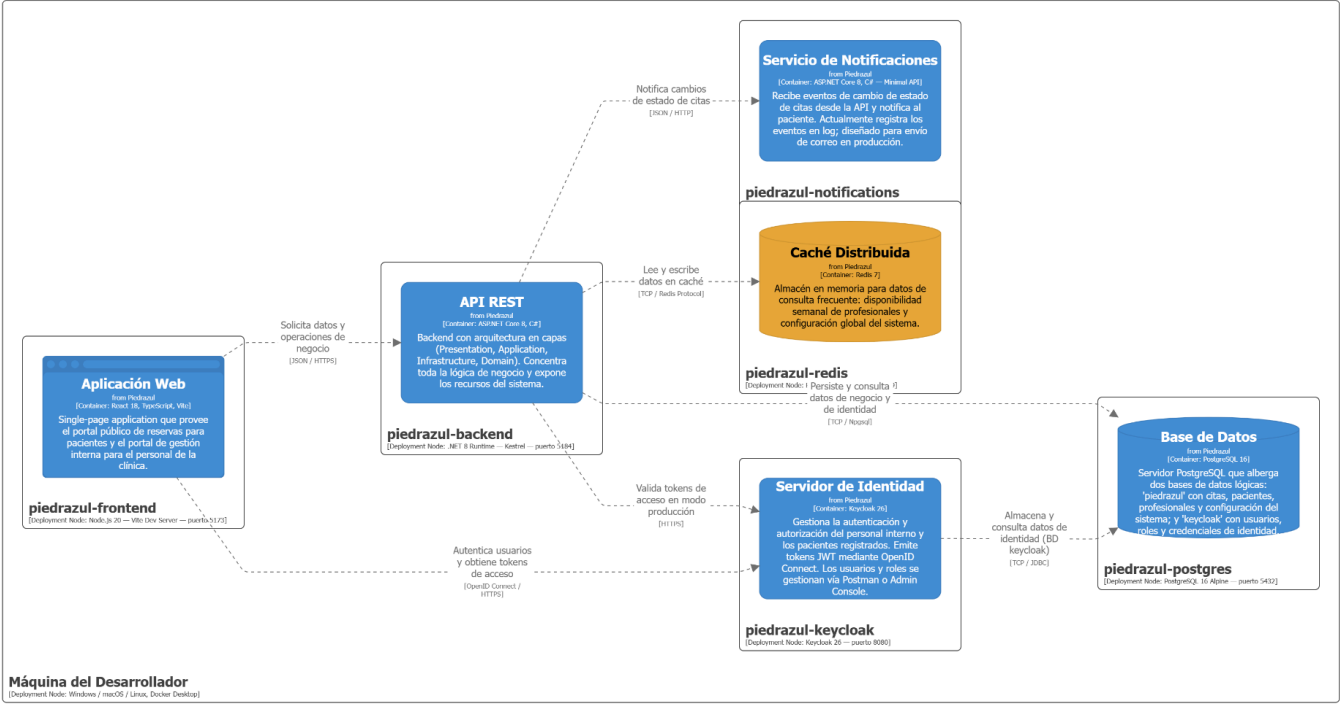
- *Servicio de Citas*: reserva, validación y exportación de reportes.
- *Servicio de Pacientes*: registro y búsqueda de perfiles.
- *Servicio de Administración*: disponibilidad semanal y ajustes globales.

Capa de Infraestructura — implementa los contratos de la capa de aplicación:

- *Repositorio de Datos*: persistencia con Entity Framework Core 8.
- *Servicio de Caché*: lectura/escritura en Redis con invalidación automática.
- *Registro de Auditoría*: trazabilidad con Serilog en archivos diarios.
- *Middleware de Autenticación*: validación JWT con soporte modo Development y Keycloak

Nota: Dado que la imagen es un poco más grande, representa ilegibilidad; colocando ruta de imagen.

[Components_API.png](#)



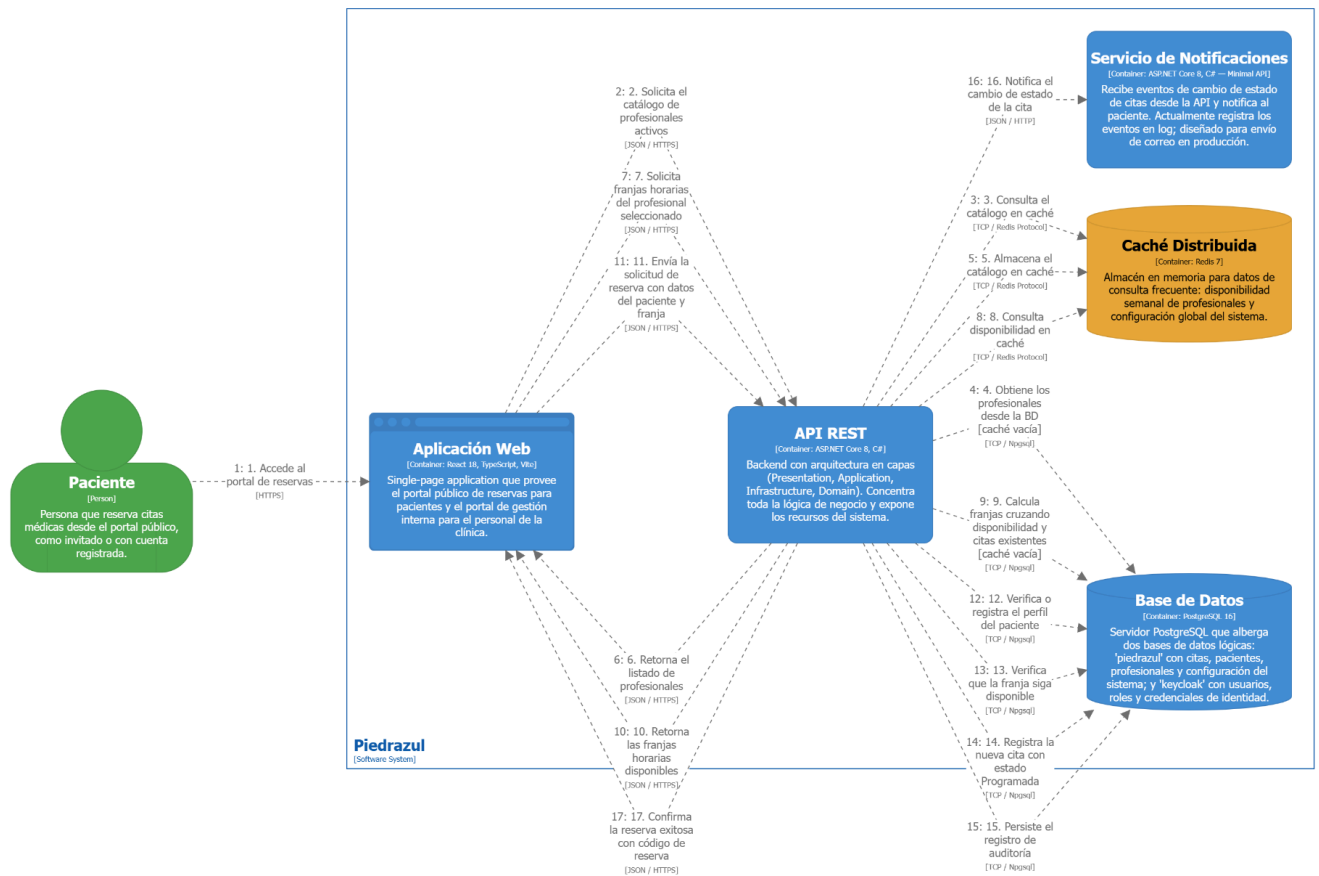
El diagrama de despliegue muestra la distribución física de los contenedores del sistema en el entorno de desarrollo mediante Docker Compose. Cada contenedor corre en un nodo independiente dentro de la máquina del desarrollador:

Nodo Docker	Tecnología	Puerto
piedrazul-frontend	Node.js 20, Vite Dev Server	5173
piedrazul-backend	.NET 8 Runtime, Kestrel	5184
piedrazul-notifications	.NET 8 Runtime, Kestrel	5185
piedrazul-redis	Redis 7 Alpine	6379
piedrazul-postgres	PostgreSQL 16 Alpine	5432
piedrazul-keycloak	Keycloak 26	8080

Nota: Dado que la imagen es un poco más grande, representa ilegibilidad; colocando ruta de imagen.

[Deployment_Dev.png](#)

8.1.6 Diagrama Dinámico 1 Paciente reserva una cita médica (RF3)

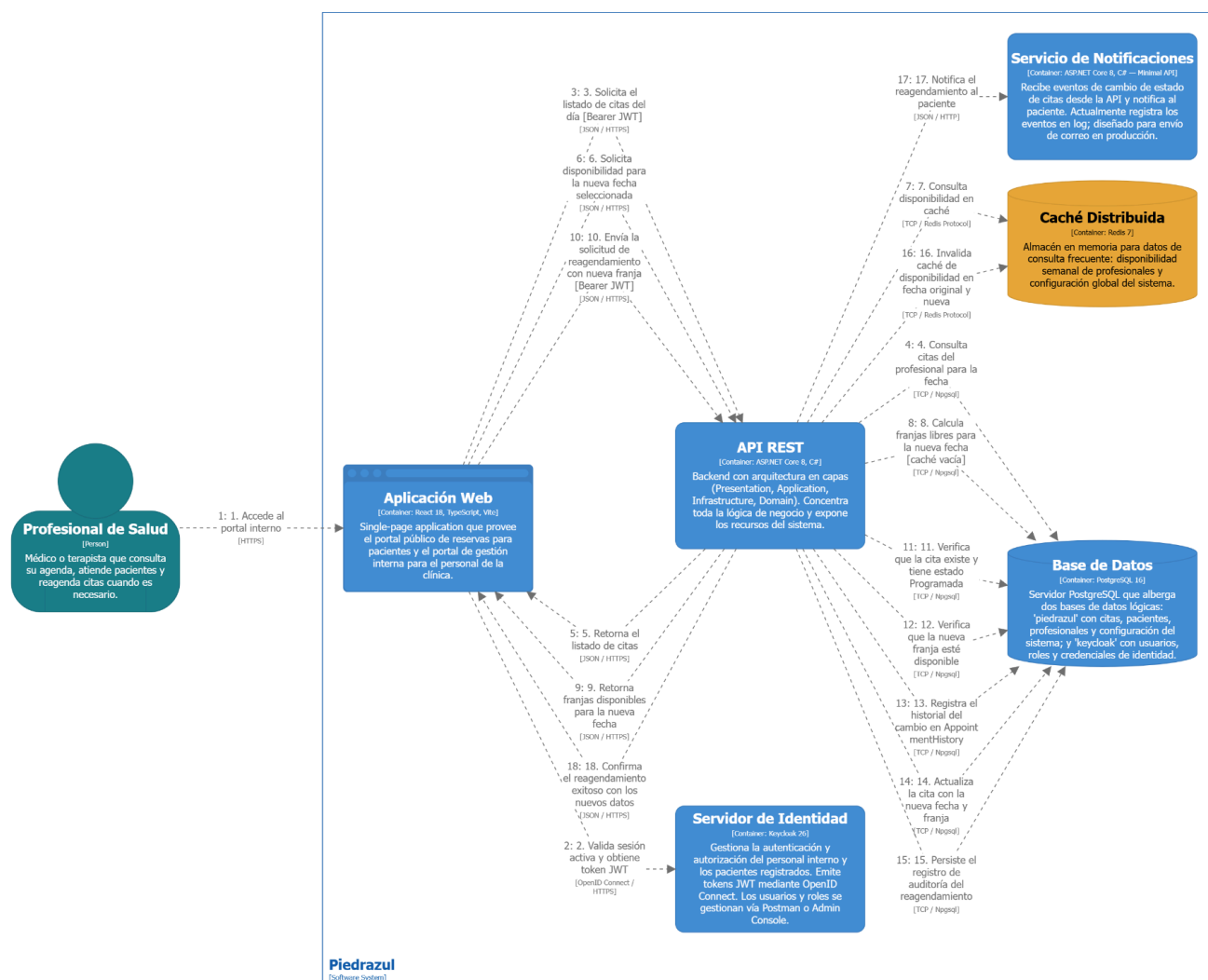


El diagrama dinámico del flujo de reserva muestra la secuencia de interacciones entre los contenedores durante el caso de uso principal del sistema. El flujo incluye: consulta con caché Redis para la disponibilidad del profesional, validación de disponibilidad de la franja en la base de datos con control de concurrencia, registro de la cita y notificación del cambio de estado al paciente.

Nota: Dado que la imagen es un poco más grande, representa ilegibilidad; colocando ruta de imagen.

[Dynamic_ReservaCita.png](#)

8.1.7 Diagrama Dinámico 2 — Reagendamento con historial (RF6)



El diagrama dinámico del reagendamento representa el flujo del Requisito Funcional 6 del tercer corte, incluido en este documento como parte del diseño prospectivo de la arquitectura. Modela la creación del registro AppointmentHistory antes de modificar la cita (atomicidad), la invalidación de caché en ambas fechas (original y nueva) y la notificación al paciente del cambio.

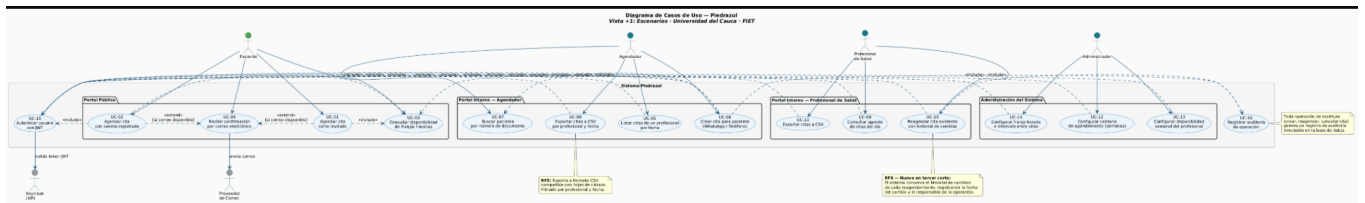
Nota: Dado que la imagen es un poco más grande, representa ilegibilidad; colocando ruta de imagen.

[Dynamic_Reagendamento.png](#)

8.2 Vistas 4+1

El framework 4+1 complementa el modelo C4 con cinco vistas que cubren diferentes perspectivas del sistema: escenarios, lógica, procesos, desarrollo y física.

Vista +1: Escenarios Diagrama de Casos de Uso



El diagrama de casos de uso representa los 16 casos de uso del sistema agrupados en cuatro paquetes funcionales: Portal Público (paciente), Portal Interno del Agendador, Portal Interno del Profesional de Salud y Administración del Sistema. Los casos de uso transversales UC-15 Autenticar usuario y UC-16 Registrar auditoría están relacionados mediante <<include>> con todos los casos de uso que requieren autenticación o trazabilidad.

Nota: Dado que la imagen es un poco más grande, representa ilegibilidad; colocando ruta de imagen.

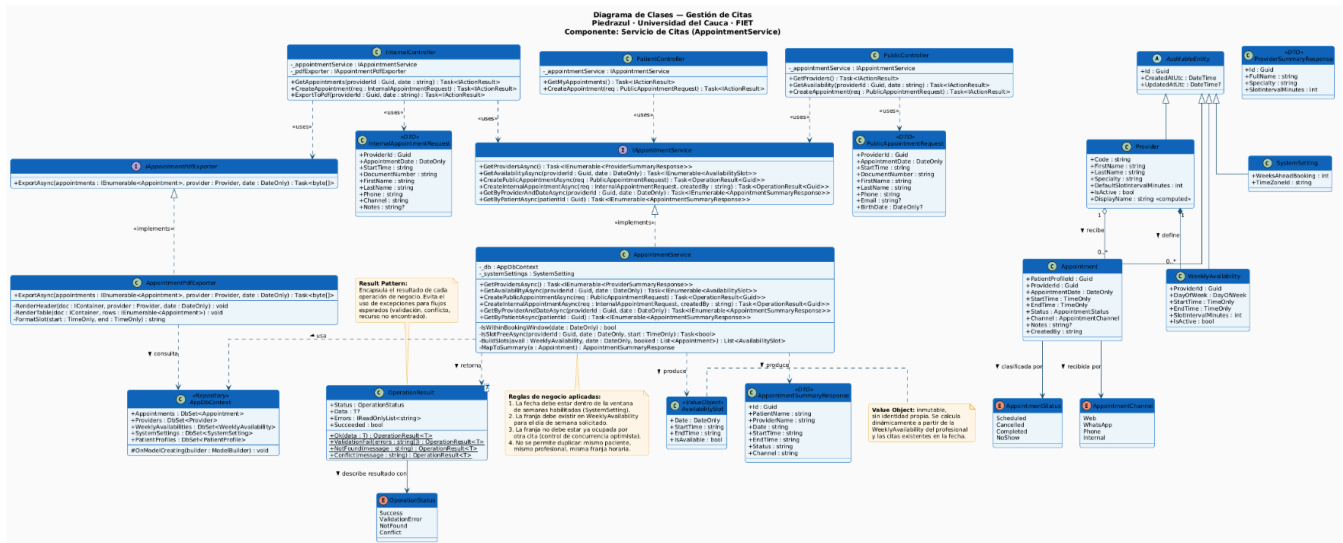
Codigo:

[V1_CasosDeUso.puml](#)

Vista:

<https://shre.ink/7GM7>

Vista Lógica Diagrama de Clases: Gestión de Citas



El diagrama de clases del componente de Gestión de Citas muestra la estructura del dominio y sus relaciones. Las entidades principales Appointment, Provider y WeeklyAvailability heredan de AuditEntity (Template Method). La interfaz IAppointmentService define el contrato del servicio, implementado por AppointmentService. Se incluye el Value Object AvailabilitySlot que encapsula la lógica de cálculo de franjas disponibles, y el patrón OperationResult<T> (Factory Method) para el manejo de resultados de operaciones de negocio.

La entidad AppointmentHistory forma parte del diseño prospectivo del RF6 del tercer corte, incluida para demostrar la capacidad de extensión de la arquitectura.

Nota: Dado que la imagen es un poco más grande, representa ilegibilidad; colocando ruta de imagen.

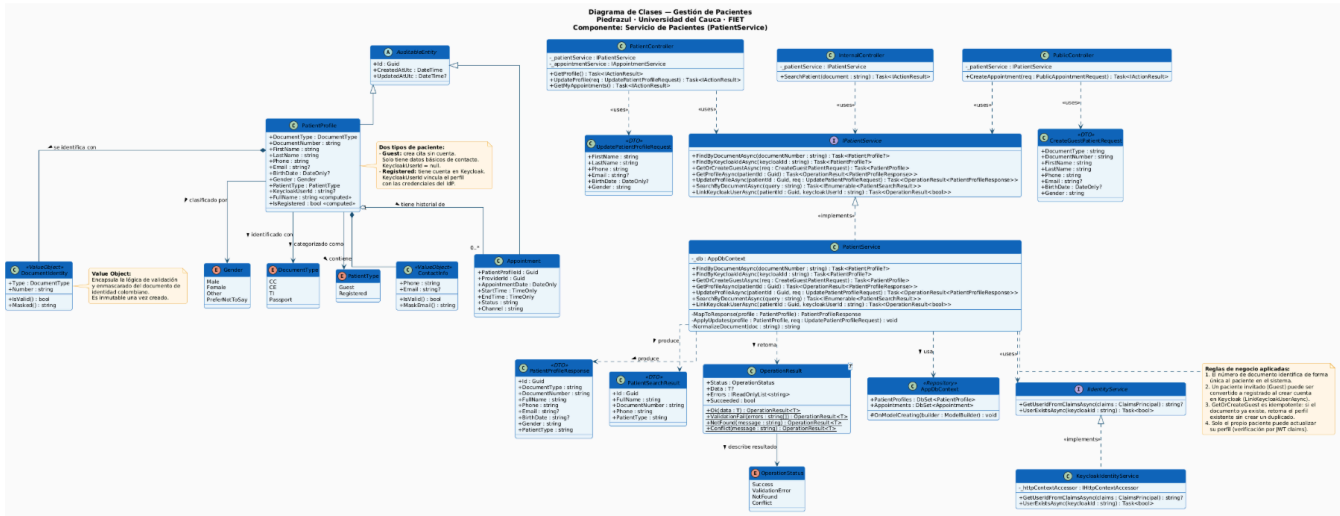
Código:

[E UML GestionCitas.puml](#)

Vista:

[PlantUML Editor](#)

Vista Lógica Diagrama de Clases: Gestión de Pacientes



El diagrama de clases del componente de Gestión de Pacientes muestra la entidad central **PatientProfile** con sus Value Objects **ContactInfo** y **DocumentIdentity**. Se distinguen dos tipos de paciente mediante el enum **PatientType**: **Guest** (reserva sin cuenta, `KeycloakUserId = null`) y **Registered** (cuenta vinculada a Keycloak). La interfaz **IService** implementada por **KeycloakIdentityService** desacopla la lógica de negocio del proveedor de identidad externo.

Nota: Dado que la imagen es un poco más grande, representa ilegibilidad; colocando ruta de imagen.

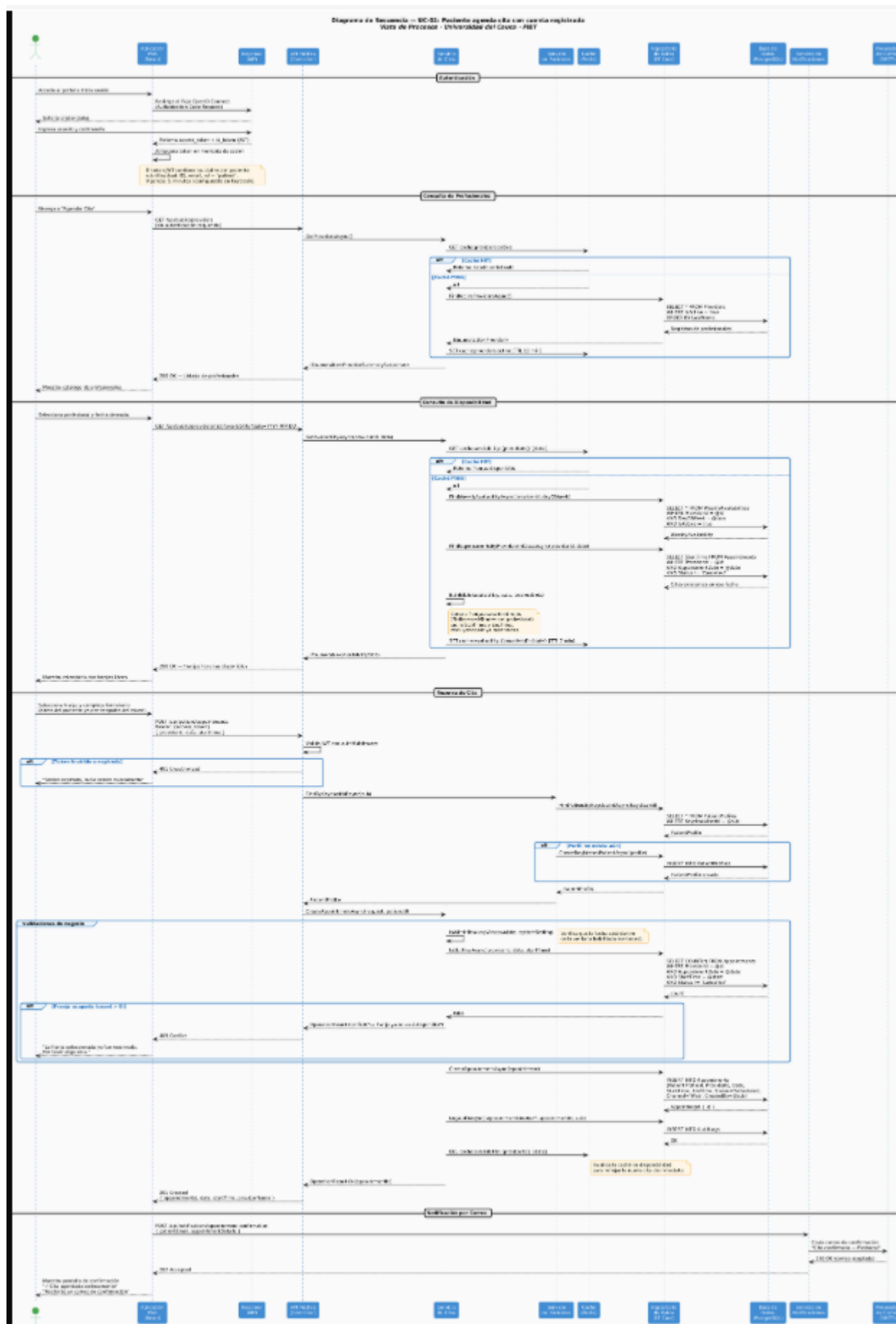
Codigo:

[F_UML_GestionPacientes.puml](#)

Vista:

[PlantUML Editor](#)

Vista de Procesos Diagrama de Secuencia: Reservar Cita



El diagrama de secuencia del flujo de reserva detalla las interacciones internas entre los componentes de la API REST durante la ejecución del caso de uso RF3. Muestra tres fases claramente diferenciadas: autenticación OIDC con Keycloak, consulta de disponibilidad con estrategia de caché (Redis primero, PostgreSQL si caché vacía), y reserva con validaciones de negocio (ventana de agendamiento, disponibilidad de franja, unicidad de reserva). Incluye los flujos alternativos para token inválido, caché miss y franja ya ocupada.

Nota: Dado que la imagen es un poco más grande, representa ilegibilidad; colocando ruta de imagen.

Codigo:

[V2_SEQ_ReservaCita.puml](#)

Vista:

[PlantUML Editor](#)

prospectivo de la arquitectura del tercer corte. Detalla la secuencia de validaciones (estado de la cita, disponibilidad de la nueva franja), la creación del registro de historial previo a la modificación (garantizando atomicidad mediante una única transacción de base de datos) y la invalidación de caché en ambas fechas.

Nota: Dado que la imagen es un poco más grande, representa ilegibilidad; colocando ruta de imagen.

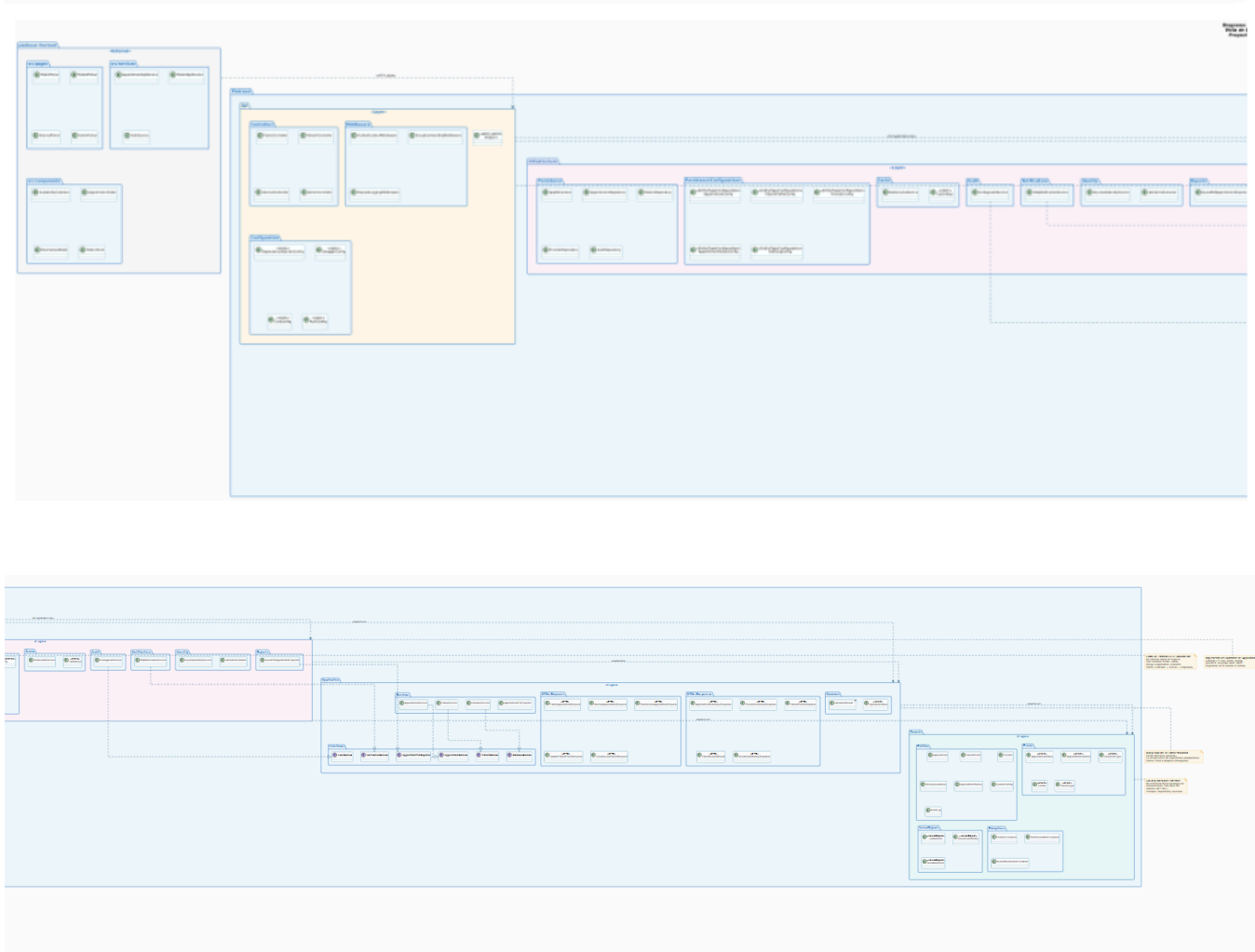
Codigo:

[V3_SEQ_Reagendamento.puml](#)

Vista:

[PlantUML Editor](#)

Vista de Desarrollo — Diagrama de Paquetes



El diagrama de paquetes representa la organización del código fuente del backend en cuatro proyectos que implementan las capas de Clean Architecture:

- **Piedrazul.Domain:** entidades, enums y value objects. Sin dependencias externas.
- **Piedrazul.Application:** interfaces (puertos), servicios y DTOs. Depende solo de Domain.
- **Piedrazul.Infrastructure:** implementaciones de los contratos de Application (adaptadores). Contiene EF Core, Redis, Serilog, Keycloak client.
- **Piedrazul.Api:** controllers, middlewares y configuración de startup. Solo conoce Application e Infrastructure (únicamente para registro de dependencias).

La regla de dependencia es unidireccional: las capas externas dependen de las internas, nunca al revés. Esto es verificable en los archivos `.csproj` del repositorio.

Nota: Dado que la imagen es un poco más grande, representa ilegibilidad; colocando ruta de imagen.

Código:

[V4 Paquetes Desarrollo.puml](#)

Vista:

[PlantUML Editor](#)

4 Listado de patrones de diseño GoF

Listado de Patrones de Diseño GoF Implementados

Patrones Creacionales

1. Factory Method

Contexto: En la capa de Application, el método estático `OperationResult<T>.Success()`, `OperationResult<T>.Validation()`, `OperationResult<T>.NotFound()` y `OperationResult<T>.Conflict()` actúan como Factory Methods. En lugar de que el código cliente instancie directamente `OperationResult<T>` con su constructor privado, cada método de fábrica crea el objeto con el estado correcto.

Aplicación en el problema: Cuando `AppointmentService` necesita retornar el resultado de crear una cita, llama a `OperationResult<AppointmentResponse>.Success(data)` o `OperationResult<AppointmentResponse>.Conflict("El slot ya fue tomado")`. El constructor es privado y nadie puede crear un resultado en estado inconsistente.

Patrones Estructurales

2. Facade

Contexto: Los controladores `PublicController`, `InternalController`, `AdminController` y `PatientController` actúan como Facades. Exponen una interfaz HTTP simplificada que oculta la complejidad interna de los servicios, repositorios, validaciones y la base de datos.

Aplicación en el problema: Cuando el agendador hace `POST /api/internal/appointments`, el controlador simplemente recibe el request y llama a `_appointmentService.CreateInternalAppointmentAsync(...)`. Todo lo que ocurre detrás — validar datos, buscar paciente, verificar slot, guardar en BD, registrar en auditoría — es invisible para quien consume la API.

3. Proxy (de protección)

Contexto: El atributo `[Authorize]` de ASP.NET Core combinado con Keycloak JWT actúa como un Proxy de protección. Intercepta cada petición HTTP antes de que llegue al método del controlador y verifica credenciales y roles.

Aplicación en el problema: Un usuario con rol `Patient` que intenta llamar a `GET /api/internal/appointments` es interceptado por el Proxy antes de que la lógica de negocio se ejecute, retornando HTTP 403 automáticamente sin que el controlador procese nada.

4. Decorator

Contexto: Los middlewares del pipeline de ASP.NET Core implementan el patrón Decorator. `ExceptionHandlerMiddleware` y `SecurityHeadersMiddleware` envuelven el procesamiento de cada petición añadiendo comportamiento transversal sin modificar los controladores.

Aplicación en el problema: `SecurityHeadersMiddleware` agrega los headers `X-Frame-Options`, `X-Content-Type-Options` y `Referrer-Policy` a cada respuesta HTTP sin que ningún controlador sepa que eso ocurre. Se "decora" la respuesta transparentemente.

Patrones de Comportamiento

5. Strategy

Contexto: El sistema de autenticación implementa Strategy a través de `DevelopmentAuthHandler` y el handler de Keycloak JWT. Ambos implementan el mismo contrato de autenticación de ASP.NET Core pero con algoritmos completamente diferentes.

Aplicación en el problema: En desarrollo local se usa `DevelopmentAuthHandler` que simula usuarios sin necesidad de Keycloak. En producción se usa el handler JWT real. El cambio se hace con una línea en `appsettings.json` (`Authentication:Mode`) sin modificar ningún controlador ni servicio.

6. Template Method

Contexto: `AppDbContext` sobrescribe el método `SaveChangesAsync()` de `DbContext` (la clase base de Entity Framework). El método base define el algoritmo de persistencia y la subclase agrega el paso de actualización de timestamps de auditoría.

Aplicación en el problema: Antes de cualquier guardado en la BD, el sistema automáticamente actualiza CreatedAtUtc y UpdatedAtUtc en todas las entidades que heredan de AuditableEntity. Ningún servicio tiene que recordar hacer esto manualmente.

7. Observer

Contexto: IAuditLogger con su implementación SerilogAuditLogger implementa el patrón Observer. Los servicios notifican eventos importantes (cita creada, estado cambiado, búsqueda de paciente) sin acoplarse al mecanismo de registro.

Aplicación en el problema: Cuando AppointmentService crea una cita exitosamente, llama a _audit.LogAppointmentCreated(...). No sabe si eso escribe en un archivo, en una base de datos o en un servicio externo. La implementación concreta de Serilog escribe en archivos rotados diariamente.

8. Null Object

Contexto: NoOpNotificationClient implementa INotificationClient con métodos que no hacen nada (return Task.CompletedTask). Es el patrón Null Object aplicado al cliente de notificaciones.

Aplicación en el problema: Durante el desarrollo, el sistema necesita un INotificationClient registrado para que la inyección de dependencias funcione, pero no se quiere enviar notificaciones reales. NoOpNotificationClient cumple el contrato sin efectos secundarios, evitando condicionales if (notificationClient != null) en el código.

En el Frontend (React)

9. Observer (Context API)

Contexto: AuthContext implementa el patrón Observer de React. Los componentes se suscriben al contexto de autenticación y se actualizan automáticamente cuando el estado de sesión cambia.

Aplicación en el problema: Cuando un paciente inicia sesión, AuthContext notifica a todos los componentes suscritos — el encabezado, las rutas protegidas, el perfil — sin necesidad de pasar props manualmente en cada nivel del árbol de componentes.

10. Proxy (ProtectedRoute)

Contexto: El componente ProtectedRoute actúa como Proxy de acceso en el frontend. Intercepta la navegación hacia rutas protegidas y verifica autenticación y rol antes de renderizar el componente destino.

Aplicación en el problema: Si un usuario sin sesión intenta navegar a /portal/interno/citas, ProtectedRoute lo redirige automáticamente a /portal/interno/login sin que InternalAppointmentsPage llegue a renderizarse.

	1.